

Curso Profesional de Entity Framework Core 8

Módulo 3 - Consultas con LINQ

TEORÍA

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 3.1 — Fundamentos de LINQ to Entities	3
Punto 3.2 — Consultas básicas: Where, OrderBy y ThenBy	7
Punto 3.3 — Proyecciones con Select y tipos anónimos	12
Punto 3.4 — Proyecciones a DTOs	18
Punto 3.5 — Consultas de agregación: Count, Sum, Average, Min y Max	24
Punto 3.6 — Agrupaciones con proyección	30
Punto 3.7 — Joins y navegación en consultas	36
Punto 3.8 — Eager Loading con Include y ThenInclude	41
Punto 3.9 — Lazy Loading: configuración, funcionamiento y riesgos	47
Punto 3.10 — Explicit Loading	53
Punto 3.11 — Composición de consultas y ejecución diferida	57
Punto 3.12 — Buenas prácticas en el acceso a datos y composición de consultas	63

Módulo 3 — Consultas con LINQ

12 puntos · 6 horas · AceriaData · .NET 8 · Entity Framework Core 8 · SQL Server LocalDB

Este módulo continúa directamente el estado validado M02/PROYECTO/2.12. Conserva Clean Architecture, la historia real de migraciones y los filtros globales heredados de M2. El contenido detallado procede de los doce puntos suministrados para M3; cuando una afirmación genérica del material no coincide con el comportamiento comprobado en EF Core 8, la explicación se ajusta al código y al E2E real del repositorio.

Punto 3.1 — Fundamentos de LINQ to Entities

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se exploran los fundamentos de LINQ to Entities en el proyecto AceriaData, diferenciando entre `IEnumerable<T>` e `IQueryable<T>`, comprendiendo la ejecución diferida y la traducción a SQL.

Objetivos de aprendizaje

- Comprender qué es LINQ y qué es LINQ to Entities.
- Diferenciar entre `IEnumerable<T>` e `IQueryable<T>`.
- Comprender la ejecución diferida y la materialización.
- Identificar qué expresiones LINQ se traducen a SQL y cuáles no.
- Analizar el SQL generado por una consulta LINQ.
- Aplicar consultas LINQ básicas al proyecto AceriaData.

Teoría

Qué es LINQ

LINQ (Language Integrated Query) es un conjunto de características del lenguaje C# que permite escribir consultas sobre colecciones de datos de forma tipada y segura. LINQ no es una tecnología de bases de datos: es una forma de expresar consultas que puede aplicarse a colecciones en memoria, a bases de datos, a documentos XML o a servicios web. La misma sintaxis se usa para todos ellos.

```
var numeros = new List<int> { 1, 2, 3, 4, 5 };  
var pares = numeros.Where(n => n % 2 == 0).ToList();
```

CSHARP

La primera línea declara una lista de números. La segunda línea filtra los números pares usando LINQ. La consulta se ejecuta en memoria porque la fuente es una lista. El resultado es una lista con los números 2 y 4.

LINQ ofrece dos sintaxis: la sintaxis de método, que usa métodos de extensión encadenados, y la sintaxis de consulta, que usa palabras clave como `from`, `where` y `select`. Ambas producen el mismo resultado.

```
// Sintaxis de método  
var pares = numeros.Where(n => n % 2 == 0).ToList();  
  
// Sintaxis de consulta  
var pares = (from n in numeros  
             where n % 2 == 0  
             select n).ToList();
```

CSHARP

La primera forma usa métodos de extensión. La segunda usa palabras clave. La sintaxis de método es la más usada en EF Core porque se compone mejor con otras operaciones.

Qué es LINQ to Entities

LINQ to Entities es la implementación de LINQ que usa EF Core para consultar bases de datos relacionales. Las consultas se escriben contra las propiedades `DbSet<T>` y se traducen a SQL por el proveedor. La consulta no se ejecuta hasta que se materializa.

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
```

CSHARP

La primera línea inicia la consulta sobre OrdenesFabricacion. La segunda filtra por cliente. La tercera materializa la consulta. Solo en la tercera línea se ejecuta el SQL contra SQL Server.

LINQ to Entities no es lo mismo que LINQ to Objects. En LINQ to Objects, las expresiones se ejecutan en memoria con delegados. En LINQ to Entities, las expresiones se traducen a SQL con árboles de expresión. Algunas expresiones que funcionan en LINQ to Objects no se pueden traducir a SQL.

IEnumerable vs IQueryable

La diferencia entre `IEnumerable<T>` e `IQueryable<T>` es fundamental para entender LINQ to Entities. `IEnumerable<T>` expone una secuencia que se recorre mediante un enumerador; por sí solo no significa que todos los datos estén ya en memoria. `IQueryable<T>` conserva una expresión de consulta y delega su ejecución a un proveedor. En `AceriaData`, después de `ToList()` los resultados sí están materializados en memoria, mientras que un `DbSet<T>` usado como `IQueryable<T>` permite al proveedor relacional de EF Core traducir a SQL las partes compatibles de la expresión.

```
IEnumerable<OrdenFabricacion> enumerable = context.OrdenesFabricacion.ToList();
IQueryable<OrdenFabricacion> queryable = context.OrdenesFabricacion;
```

CSHARP

La primera línea materializa la consulta en una lista y la asigna a `IEnumerable`. La segunda línea mantiene la consulta como `IQueryable` sin ejecutarla.

En este ejemplo concreto, el `IEnumerable<T>` procede de un `ToList()`, por lo que los filtros posteriores se ejecutan con LINQ to Objects sobre los datos ya cargados. La consulta que parte del `DbSet<T>` continúa como `IQueryable<T>` y el proveedor de EF Core puede traducir el `Where` compatible a SQL antes de materializar.

```
// Filtro en memoria: se cargan todas las órdenes y se filtran en C#
var ordenes1 = context.OrdenesFabricacion.ToList().Where(o => o.Cliente == "Constructora del Norte");

// Filtro en el servidor: se filtra en SQL
var ordenes2 = context.OrdenesFabricacion.Where(o => o.Cliente == "Constructora del Norte").ToList();
```

CSHARP

La primera línea carga todas las órdenes en memoria y después filtra. La segunda línea filtra en SQL y solo carga las órdenes que cumplen la condición. La segunda es mucho más eficiente porque transfiere menos datos.

Error común. si se llama a `ToList` antes de aplicar los filtros, se carga toda la tabla en memoria y se pierde la ventaja de la traducción a SQL. Se debe aplicar `ToList` al final de la consulta, después de todos los filtros y proyecciones.

Ejecución diferida

La ejecución diferida es el comportamiento por el que una consulta LINQ no se ejecuta hasta que se itera sobre ella o se materializa. Esto permite componer consultas de forma incremental sin ejecutar nada hasta el final.

```
var consulta = context.OrdenesFabricacion.AsQueryable();

if (!string.IsNullOrEmpty(cliente))
{
    consulta = consulta.Where(o => o.Cliente == cliente);
}

if (fechaDesde.HasValue)
{
    consulta = consulta.Where(o => o.FechaCreacion >= fechaDesde.Value);
}

var resultados = consulta.ToList();
```

CSHARP

La primera línea inicia la consulta. Las siguientes líneas añaden filtros condicionalmente. La última línea materializa la consulta. Solo se ejecuta un `SELECT` con todos los filtros aplicados.

La ejecución diferida es útil cuando los filtros dependen de parámetros opcionales. También es útil cuando se quiere reutilizar una consulta base con diferentes filtros.

Error común. materializar antes de terminar la composición corta la consulta de servidor: los operadores aplicados después sobre la lista se ejecutan en memoria. Además, cada operador terminal que se invoque de nuevo sobre el `IQueryable` original supone una nueva ejecución. Conviene componer primero y ejecutar al final.

Materialización

Una consulta diferida termina cuando se enumera o se invoca un operador terminal. `ToList()` y `ToArray()` ejecutan la consulta y materializan una colección; `FirstOrDefault()` o `SingleOrDefault()` ejecutan la consulta y obtienen un elemento; `Count()`, `Any()` o `Sum()` ejecutan una operación escalar en el proveedor cuando se aplican al `IQueryable`. No todos estos operadores materializan una colección, aunque todos fuerzan la ejecución de esa consulta.

```
var lista = consulta.ToList();
var primera = consulta.FirstOrDefault();
var total = consulta.Count();
var existe = consulta.Any();
```

CSHARP

La primera línea materializa la consulta en una lista. La segunda obtiene el primer elemento o null. La tercera cuenta los elementos. La cuarta comprueba si hay al menos un elemento. En este fragmento cada llamada parte del mismo `IQueryable` y cada operador terminal se ejecuta de forma independiente.

Error común. `var lista = consulta.ToList(); var total = lista.Count;` ejecuta una sola consulta de base de datos y cuenta después en memoria. Si solo se necesita el número de filas, `consulta.Count()` evita transferirlas. Solo habría una segunda consulta si, después de `ToList()`, se volviera a ejecutar por separado `consulta.Count()` sobre el `IQueryable` original.

Qué expresiones se traducen a SQL

No todas las expresiones LINQ se traducen a SQL. El proveedor de EF Core conoce los patrones que puede traducir. Si una expresión no traducible aparece en una parte de la consulta que debe ejecutarse en el servidor —por ejemplo, un filtro— EF Core 8 normalmente lanza una excepción. La proyección superior tiene reglas distintas y puede admitir evaluación cliente para elementos que no necesitan ejecutarse en SQL.

```
// Se traduce a SQL
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Cliente.StartsWith("Constructora"))
    .ToList();

// No se traduce a SQL: se ejecuta en memoria
var ordenes2 = context.OrdenesFabricacion
    .ToList()
    .Where(o => MiMetodoPersonalizado(o.Cliente))
    .ToList();
```

CSHARP

La primera consulta se traduce a SQL porque `StartsWith` tiene equivalente en SQL (`LIKE 'Constructora%'`). La segunda consulta no se traduce porque `MiMetodoPersonalizado` no tiene equivalente en SQL. La primera línea materializa toda la tabla y la segunda filtra en memoria.

Error común. colocar un método personalizado no traducible dentro de un filtro u otra parte que deba ejecutarse en el servidor provoca un fallo de traducción. Si se desea continuar en memoria, la frontera debe hacerse explícita con `AsEnumerable()` o una materialización consciente del coste; una proyección superior puede tener un tratamiento distinto.

Operadores más habituales

Los operadores LINQ más habituales en EF Core son `Where`, `OrderBy`, `ThenBy`, `Select`, `SelectMany`, `Distinct`, `GroupBy`, `Join`, `Include`, `Skip`, `Take`, `FirstOrDefault`, `SingleOrDefault`, `Any`, `All`, `Count`, `Sum`, `Min`, `Max`, `Average`.

```
var resultado = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Select(o => new { o.NumeroOrden, o.Cliente });
```

CSHARP

```
.Take(10)
.ToList();
```

La primera línea inicia la consulta. La segunda filtra por estado. La tercera ordena por fecha. La cuarta proyecta a un tipo anónimo. La quinta limita a diez resultados. La sexta materializa la consulta.

Error común. si se aplica Take antes de OrderBy, el resultado es indeterminado porque el orden no está definido. Se debe aplicar OrderBy antes de Take para obtener un resultado determinista.

Análisis del SQL generado

EF Core permite ver el SQL generado por una consulta con el método ToQueryString. Este método devuelve la sentencia SQL sin ejecutarla. Es útil para diagnosticar problemas de rendimiento y para entender cómo EF Core traduce las consultas.

```
var consulta = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte")
    .OrderBy(o => o.FechaCreacion);

var sql = consulta.ToQueryString();
Console.WriteLine(sql);
```

CSHARP

La primera línea inicia la consulta. La segunda filtra por cliente. La tercera ordena por fecha. La cuarta obtiene el SQL. La quinta lo imprime. La consulta no se ejecuta.

El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] = N'Constructora del Norte'
ORDER BY [o].[FechaCreacion]
```

SQL

La primera línea selecciona las columnas. La segunda indica la tabla. La tercera filtra por cliente. La cuarta ordena por fecha.

Error común. si el SQL generado no incluye el filtro esperado, se debe revisar la consulta LINQ. Puede que el filtro se esté aplicando en memoria después de materializar.

El proyecto AceriaData

En el proyecto AceriaData, el Módulo 3 se centra en las consultas LINQ. Los casos de uso de la capa de aplicación usarán los repositorios para ejecutar consultas que aprovechen las capacidades de LINQ. En este punto se exploran los fundamentos: IEnumerable vs IQueryable, ejecución diferida, materialización, traducción a SQL y análisis del SQL generado. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Validación técnica en AceriaData

En el checkpoint M03/PROYECTO/3.1, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar Database.Migrate(). La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre IEnumerable<T> frente a IQueryable<T>, ejecución diferida, materialización, ToQueryString(), el seam IQueryable se mantiene temporalmente con finalidad docente y se elimina del puerto en 3.12. El punto termina con el marcador E2E 3.1 OK.

Resumen de la teoría

- LINQ es un conjunto de características de C# para consultar colecciones.
- LINQ to Entities traduce consultas LINQ a SQL.
- IEnumerable<T> expone una secuencia; en el ejemplo, tras ToList() sus datos ya están en memoria.
- IQueryable<T> conserva una expresión que un proveedor puede interpretar; el proveedor relacional de EF Core traduce a SQL las expresiones compatibles.

- La ejecución diferida pospone la ejecución hasta la enumeración o un operador terminal.
- `ToList()` materializa una colección; operadores como `FirstOrDefault()`, `Count()` o `Any()` también fuerzan la ejecución y devuelven un elemento o un escalar.
- No todas las expresiones LINQ se traducen a SQL.
- Los operadores más habituales son `Where`, `OrderBy`, `Select`, `GroupBy`, `Join`, `Include`, `Skip`, `Take`.
- `ToQueryString` permite ver el SQL generado sin ejecutarlo.
- En el proyecto `AceriaData` se aplican estos fundamentos a las consultas del dominio.

Punto 3.2 — Consultas básicas: Where, OrderBy y ThenBy

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se añaden consultas básicas con filtros y ordenaciones al proyecto `AceriaData`, encapsulándolas en métodos específicos del repositorio para evitar exponer `IQueryable` fuera de la infraestructura.

Objetivos de aprendizaje

- Comprender el operador `Where` y sus variantes.
- Comprender el operador `OrderBy` y `OrderByDescending`.
- Comprender el operador `ThenBy` y `ThenByDescending`.
- Combinar filtros y ordenaciones en una misma consulta.
- Analizar el SQL generado por las consultas con filtros y ordenaciones.
- Encapsular las consultas en métodos del repositorio.
- Aplicar estas consultas al proyecto `AceriaData`.

Teoría

El operador Where

El operador `Where` filtra los elementos de una secuencia que cumplen una condición. La condición se expresa como una expresión lambda que devuelve un booleano. EF Core traduce la condición a una cláusula `WHERE` en SQL. Solo se devuelven los elementos que cumplen la condición.

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
```

CSHARP

La primera línea inicia la consulta sobre `OrdenesFabricacion`. La segunda línea filtra por cliente. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] = N'Constructora del Norte'
```

SQL

La primera línea selecciona las columnas. La segunda indica la tabla. La tercera filtra por cliente. Solo se devuelven las órdenes del cliente indicado.

Condiciones compuestas con Where

El operador `Where` acepta condiciones compuestas con los operadores lógicos `&&` (AND) y `||` (OR). La condición se traduce a SQL con los operadores `AND` y `OR`.

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte" && o.Estado == "Pendiente")
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por cliente y estado. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[Estado] = N'Pendiente'
```

SQL

La cláusula WHERE incluye dos condiciones unidas por AND. Solo se devuelven las órdenes que cumplen ambas condiciones.

Condiciones con OR

El operador || se traduce a OR en SQL. Se usa cuando se quiere que se cumpla al menos una de las condiciones.

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente" || o.Estado == "EnProceso")
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado pendiente o en proceso. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' OR [o].[Estado] = N'EnProceso'
```

SQL

La cláusula WHERE incluye dos condiciones unidas por OR. Se devuelven las órdenes que cumplen al menos una de las dos condiciones.

Condiciones con métodos de cadena

EF Core traduce varios métodos de cadena a SQL. Los más habituales son StartsWith, EndsWith, Contains, ToLower y ToUpper.

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Cliente.StartsWith("Constructora"))
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por clientes que empiezan por "Constructora". La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] LIKE N'Constructora%'
```

SQL

StartsWith se traduce a LIKE 'Constructora%'. EndsWith se traduce a LIKE '%Constructora'. Contains se traduce a LIKE '%Constructora%'.

Error común. si se usa ToLower() o ToUpper() dentro de una consulta, EF Core los traduce a las funciones LOWER y UPPER de SQL Server. Sin embargo, si se usan en combinación con StartsWith, pueden provocar que el índice no se use. Se debe evitar cuando sea posible.

Condiciones con fechas

Las fechas se filtran con los operadores de comparación habituales: >, >=, <, <=, ==. EF Core traduce estas comparaciones a SQL.

```
var fechaDesde = new DateTime(2024, 1, 1);
var ordenes = context.OrdenesFabricacion
    .Where(o => o.FechaCreacion >= fechaDesde)
    .ToList();
```

CSHARP

La primera línea declara la fecha desde. La segunda línea inicia la consulta. La tercera línea filtra por fecha de creación mayor o igual que la fecha desde. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[FechaCreacion] >= @_fechaDesde_0
```

SQL

La cláusula WHERE compara la fecha de creación con el parámetro. El parámetro se pasa de forma segura para evitar inyección SQL.

El operador OrderBy

El operador OrderBy ordena los elementos de una secuencia de forma ascendente según una clave. La clave se expresa como una expresión lambda que devuelve el valor por el que se ordena. EF Core traduce la ordenación a una cláusula ORDER BY en SQL.

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea ordena por fecha de creación ascendente. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
ORDER BY [o].[FechaCreacion]
```

SQL

La cláusula ORDER BY ordena por fecha de creación. Si no se especifica ASC o DESC, el orden es ascendente por defecto.

El operador OrderByDescending

El operador OrderByDescending ordena los elementos de forma descendente según una clave. Se traduce a ORDER BY ... DESC en SQL.

```
var ordenes = context.OrdenesFabricacion
    .OrderByDescending(o => o.FechaCreacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea ordena por fecha de creación descendente. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
ORDER BY [o].[FechaCreacion] DESC
```

SQL

La cláusula ORDER BY incluye DESC para indicar el orden descendente.

El operador ThenBy

El operador ThenBy añade un criterio de ordenación secundario a una ordenación existente. Se usa después de OrderBy o OrderByDescending. Se traduce a una segunda columna en la cláusula ORDER BY.

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.Cliente)
    .ThenBy(o => o.FechaCreacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea ordena por cliente ascendente. La tercera línea añade un criterio secundario por fecha de creación ascendente. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
ORDER BY [o].[Cliente], [o].[FechaCreacion]
```

SQL

La cláusula ORDER BY incluye dos columnas separadas por coma. Primero ordena por cliente y, dentro de cada cliente, por fecha de creación.

El operador ThenByDescending

El operador ThenByDescending añade un criterio de ordenación secundario descendente. Se traduce a una segunda columna en la cláusula ORDER BY con DESC.

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.Cliente)
    .ThenByDescending(o => o.FechaCreacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea ordena por cliente ascendente. La tercera línea añade un criterio secundario por fecha de creación descendente. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
ORDER BY [o].[Cliente], [o].[FechaCreacion] DESC
```

SQL

La cláusula ORDER BY incluye dos columnas: Cliente ascendente y FechaCreacion descendente.

Combinar Where, OrderBy y ThenBy

Los operadores se combinan en una misma consulta. El orden de las llamadas importa: primero se filtran los datos con Where, después se ordenan con OrderBy y ThenBy.

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.Cliente)
    .ThenByDescending(o => o.FechaCreacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por cliente ascendente. La cuarta línea añade un criterio secundario por fecha descendente. La quinta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente'
ORDER BY [o].[Cliente], [o].[FechaCreacion] DESC
```

SQL

La cláusula WHERE filtra por estado. La cláusula ORDER BY ordena por cliente y fecha. El orden de las cláusulas en SQL es SELECT, FROM, WHERE, ORDER BY.

El orden de las cláusulas en SQL

SQL tiene un orden fijo para las cláusulas: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, OFFSET, FETCH. EF Core genera las cláusulas en este orden independientemente del orden de las llamadas LINQ.

```
// Orden LINQ: Where -> OrderBy -> Select
var consulta = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Select(o => new { o.NumeroOrden, o.Cliente });

// Orden SQL: SELECT -> FROM -> WHERE -> ORDER BY
// SELECT [o].[NumeroOrden], [o].[Cliente]
// FROM [OrdenesFabricacion] AS [o]
// WHERE [o].[Estado] = N'Pendiente'
// ORDER BY [o].[FechaCreacion]
```

CSHARP

La primera línea inicia la consulta. Las siguientes líneas añaden filtro, ordenación y proyección. El SQL generado coloca las cláusulas en el orden correcto, independientemente del orden de las llamadas LINQ.

Error común. pensar que Select antes de Where impide por sí mismo la traducción. Si el predicado solo usa miembros que siguen disponibles en la proyección, EF Core puede seguir traduciendo la consulta. Filtrar antes de proyectar suele ser más claro y evita perder propiedades necesarias para filtros posteriores, pero hay que revisar el SQL real.

Ordenación por múltiples columnas

Se pueden encadenar varios `ThenBy` y `ThenByDescending` para ordenar por múltiples columnas. Cada uno añade un criterio de ordenación.

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.Estado)
    .ThenBy(o => o.Cliente)
    .ThenByDescending(o => o.FechaCreacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea ordena por estado. La tercera línea añade un criterio por cliente. La cuarta línea añade un criterio por fecha descendente. La quinta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
ORDER BY [o].[Estado], [o].[Cliente], [o].[FechaCreacion] DESC
```

SQL

La cláusula `ORDER BY` incluye tres columnas. Primero ordena por estado, después por cliente y finalmente por fecha descendente.

Encapsular consultas en el repositorio

Para evitar exponer `IQueryable` fuera de la infraestructura, las consultas se encapsulan en métodos específicos del repositorio. La capa de aplicación llama a estos métodos sin conocer los detalles de LINQ.

```
public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta);
}
```

CSHARP

La primera línea declara la interfaz. La segunda línea declara el método que obtiene las órdenes pendientes de un cliente. La tercera línea declara el método que obtiene las órdenes por estado ordenadas por fecha. La cuarta línea declara el método que obtiene las órdenes por rango de fechas.

La implementación usa LINQ internamente.

```
public List<OrdenFabricacion> ObtenerPendientesPorCliente(string cliente)
{
    return _context.OrdenesFabricacion
        .Where(o => o.Cliente == cliente && o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado)
{
    return _context.OrdenesFabricacion
        .Where(o => o.Estado == estado)
        .OrderByDescending(o => o.FechaCreacion)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorRangoDeFechas(DateTime desde, DateTime hasta)
{
    return _context.OrdenesFabricacion
        .Where(o => o.FechaCreacion >= desde && o.FechaCreacion <= hasta)
        .OrderBy(o => o.FechaCreacion)
        .ToList();
}
```

CSHARP

La primera implementación filtra por cliente y estado, y ordena por fecha. La segunda filtra por estado y ordena por fecha descendente. La tercera filtra por rango de fechas y ordena por fecha ascendente. Todas devuelven listas materializadas.

Error común. si el repositorio devuelve `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Esto rompe la abstracción. Se deben devolver listas o entidades concretas.

El proyecto AceriaData

En el proyecto AceriaData, se añaden métodos específicos al repositorio de órdenes para encapsular las consultas con filtros y ordenaciones. La capa de aplicación usa estos métodos a través de la unidad de trabajo. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Validación técnica en AceriaData

En el checkpoint M03/PROYECTO/3.2, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `Where`, `OrderBy` / `OrderByDescending`, `ThenBy` / `ThenByDescending`, filtros por rango de fechas, SQL observable. El punto termina con el marcador E2E 3.2 OK.

Resumen de la teoría

- `Where` filtra los elementos que cumplen una condición.
- Las condiciones se combinan con `&&` y `||`.
- Los métodos de cadena `StartsWith`, `EndsWith` y `Contains` se traducen a `LIKE`.
- Las fechas se comparan con `>`, `>=`, `<`, `<=`, `==`.
- `OrderBy` ordena de forma ascendente.
- `OrderByDescending` ordena de forma descendente.
- `ThenBy` añade un criterio secundario ascendente.
- `ThenByDescending` añade un criterio secundario descendente.
- El orden de las cláusulas SQL es fijo: `SELECT`, `FROM`, `WHERE`, `ORDER BY`.
- Las consultas se encapsulan en métodos del repositorio para evitar exponer `IQueryable`.
- En el proyecto AceriaData se añaden métodos específicos al repositorio de órdenes.

Punto 3.3 — Proyecciones con Select y tipos anónimos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se añaden proyecciones con `Select` al proyecto AceriaData, incluyendo tipos anónimos y proyecciones parciales, encapsuladas en métodos del repositorio.

Objetivos de aprendizaje

- Comprender qué es una proyección y por qué es útil.
- Proyectar a tipos anónimos con `Select`.
- Proyectar a propiedades individuales.
- Proyectar a tipos anónimos con varias propiedades.
- Analizar el SQL generado por una proyección.
- Comparar el rendimiento de proyectar versus cargar entidades completas.
- Aplicar proyecciones al proyecto AceriaData.

Teoría

Qué es una proyección

Una proyección es la operación de transformar cada elemento de una secuencia en una nueva forma. En LINQ, la proyección se realiza con el operador `Select`. La proyección permite seleccionar solo las propiedades que se necesitan, en lugar de cargar la entidad completa.

```
var numeros = new List<int> { 1, 2, 3, 4, 5 };
var cuadrados = numeros.Select(n => n * n).ToList();
```

CSHARP

La primera línea declara una lista de números. La segunda línea proyecta cada número a su cuadrado. El resultado es una lista con los valores 1, 4, 9, 16, 25. La proyección transforma cada elemento de la secuencia original en un nuevo elemento.

En EF Core, la proyección tiene un impacto directo en el SQL generado. Sin proyección, EF Core genera un SELECT con todas las columnas de la entidad. Con proyección, EF Core genera un SELECT solo con las columnas proyectadas. Esto reduce el volumen de datos transferidos y mejora el rendimiento.

Proyectar a un tipo anónimo

Un tipo anónimo es un tipo que se define en el momento de la proyección y que no tiene nombre. Se crea con la palabra clave `new` seguida de las propiedades que se quieren incluir. EF Core traduce la proyección a un SELECT con las columnas correspondientes.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta sobre `OrdenesFabricacion`. La segunda línea proyecta cada orden a un tipo anónimo con las propiedades `NumeroOrden`, `Cliente` y `Estado`. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
```

SQL

La cláusula SELECT solo incluye las tres columnas proyectadas. Las columnas `Id`, `FechaCreacion`, `FechaEntrega`, `Observaciones`, `IsDeleted` y `DeletedAt` no se seleccionan. Esto reduce el volumen de datos transferidos.

Error común. si se proyecta a un tipo anónimo y se intenta usar el resultado fuera del método donde se creó, el compilador no puede inferir el tipo. Los tipos anónimos solo se pueden usar dentro del ámbito donde se declaran. Para usarlos fuera, se debe proyectar a un DTO con nombre.

Proyectar a propiedades individuales

Se puede proyectar a una sola propiedad de la entidad. El resultado es una secuencia de valores del tipo de la propiedad.

```
var clientes = context.OrdenesFabricacion
    .Select(o => o.Cliente)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden a su propiedad `Cliente`. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
```

SQL

La cláusula SELECT solo incluye la columna `Cliente`. El resultado es una lista de cadenas con los nombres de los clientes. Si hay clientes repetidos, aparecen repetidos en la lista.

Proyectar con Distinct

El operador `Distinct` elimina los duplicados de una secuencia. Se combina con `Select` para obtener valores únicos.

```
var clientesUnicos = context.OrdenesFabricacion
    .Select(o => o.Cliente)
    .Distinct()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden a su propiedad Cliente. La tercera línea elimina los duplicados. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT DISTINCT [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
```

SQL

La cláusula SELECT incluye DISTINCT para eliminar los duplicados. El resultado es una lista de clientes únicos.

Proyectar a tipos anónimos con varias propiedades

Se pueden proyectar varias propiedades a un tipo anónimo. El tipo anónimo puede incluir propiedades calculadas, propiedades renombradas y propiedades anidadas.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        Numero = o.NumeroOrden,
        Cliente = o.Cliente,
        Anio = o.FechaCreacion.Year,
        Mes = o.FechaCreacion.Month,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden a un tipo anónimo con propiedades renombradas y calculadas. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden] AS [Numero], [o].[Cliente], DATEPART(year, [o].[FechaCreacion]) AS [Anio], DATEPART(month,
[o].[FechaCreacion]) AS [Mes], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]
```

SQL

La cláusula SELECT incluye las columnas con alias y las funciones DATEPART para extraer el año y el mes. El resultado es una lista de tipos anónimos con las propiedades Numero, Cliente, Anio, Mes y Estado.

Error común. si una expresión no puede traducirse en una parte de la consulta que debe ejecutarse en el servidor, EF Core 8 normalmente lanza una excepción; no hace evaluación cliente implícita del filtro. Para continuar en memoria hay que cambiar explícitamente a LINQ to Objects con `AsEnumerable()` o materializar con `ToList()`, asumiendo el coste.

Proyectar a un DTO con nombre

Un DTO (Data Transfer Object) es una clase con nombre que se usa para transferir datos entre capas. Se proyecta a un DTO cuando se necesita usar el resultado fuera del método donde se creó o cuando se quiere una estructura de datos más compleja.

```
public class OrdenResumenDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
}

var resultado = context.OrdenesFabricacion
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea declara la clase OrdenResumenDto. Las siguientes líneas declaran sus propiedades. La penúltima línea proyecta cada orden a un OrdenResumenDto. La última línea materializa la consulta. El SQL generado es el mismo que para un tipo anónimo con las mismas propiedades.

La ventaja del DTO es que tiene nombre y se puede usar fuera del método. La desventaja es que requiere escribir más código. En proyectos con arquitectura limpia, los DTOs se colocan en la capa de aplicación.

Proyectar a un DTO con constructor

Se puede proyectar a un DTO con constructor si sus argumentos se obtienen de la consulta. EF Core selecciona del servidor los datos que necesita y construye el DTO al materializar el resultado; la proyección superior también puede contener partes evaluadas en el cliente cuando no necesitan ejecutarse en SQL.

```
public class OrdenResumenDto
{
    public string NumeroOrden { get; }
    public string Cliente { get; }
    public string Estado { get; }

    public OrdenResumenDto(string numeroOrden, string cliente, string estado)
    {
        NumeroOrden = numeroOrden;
        Cliente = cliente;
        Estado = estado;
    }
}

var resultado = context.OrdenesFabricacion
    .Select(o => new OrdenResumenDto(o.NumeroOrden, o.Cliente, o.Estado))
    .ToList();
```

CSHARP

La primera línea declara el DTO con constructor. Las siguientes líneas declaran las propiedades y el constructor. La penúltima línea proyecta cada orden a un OrdenResumenDto usando el constructor. La última línea materializa la consulta.

Error común. asumir que toda lógica de un DTO debe traducirse a SQL. EF Core permite evaluación cliente en la proyección superior, pero las expresiones usadas en filtros, ordenaciones u otras partes que deban ejecutarse en el servidor sí tienen que ser traducibles. Conviene mantener los DTO simples para que el coste y la frontera cliente/servidor sean evidentes.

Proyectar a un tipo anónimo con propiedades anidadas

Se puede proyectar a un tipo anónimo que contiene otro tipo anónimo como propiedad. Esto permite agrupar propiedades relacionadas.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        Fechas = new
        {
            Creacion = o.FechaCreacion,
            Entrega = o.FechaEntrega
        }
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden a un tipo anónimo con una propiedad anidada Fechas. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[FechaEntrega]
FROM [OrdenesFabricacion] AS [o]
```

SQL

La cláusula SELECT incluye todas las columnas proyectadas, incluyendo las de la propiedad anidada. El resultado es una lista de tipos anónimos con las propiedades NumeroOrden, Cliente y Fechas.

El impacto de la proyección en el rendimiento

La proyección reduce el volumen de datos transferidos entre la base de datos y la aplicación. Esto mejora el rendimiento en dos aspectos: menos datos que transferir y menos objetos que materializar. En tablas con muchas columnas o con columnas de tipo nvarchar(max), la diferencia puede ser significativa.

```
// Sin proyección: carga todas las columnas
var ordenes = context.OrdenesFabricacion.ToList();
```

CSHARP

```
// Con proyección: carga solo las columnas necesarias
var resmenes = context.OrdenesFabricacion
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();
```

La primera línea carga todas las columnas de todas las órdenes. La segunda línea carga solo las columnas NumeroOrden y Cliente. Si la tabla tiene diez columnas, la proyección reduce el volumen de datos en un ochenta por ciento.

Error común. si se proyecta a un tipo anónimo y después se accede a una propiedad que no está en la proyección, el compilador lanza un error. Se debe proyectar todas las propiedades que se van a usar.

Proyectar con OrderBy y Where

La proyección se puede combinar con Where y OrderBy. El orden de las llamadas importa: primero se filtra, después se ordena y finalmente se proyecta.

```
var resultado = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por fecha. La cuarta línea proyecta a un tipo anónimo. La quinta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente'
ORDER BY [o].[FechaCreacion]
```

SQL

La cláusula SELECT solo incluye las columnas proyectadas. La cláusula WHERE filtra por estado. La cláusula ORDER BY ordena por fecha. El orden de las cláusulas es el correcto.

Error común. tratar el orden Where/Select como una regla de rendimiento absoluta. EF Core puede traducir un filtro posterior si la proyección conserva los datos necesarios. Filtrar antes de proyectar suele facilitar la lectura y la composición, pero el criterio definitivo es el SQL generado.

Proyectar con navegación

Se puede proyectar una propiedad de navegación para obtener datos relacionados sin materializar la entidad completa. La forma SQL depende del *shape* de la consulta y del proveedor: puede usar JOIN, subconsultas u otras construcciones.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        Planchas = o.Planchas
            .Select(p => new { p.Espesor, p.Peso })
            .ToList()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden a un tipo anónimo con una colección de planchas proyectadas. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso], [p].[Id]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
ORDER BY [o].[Id], [p].[Id]
```

SQL

La cláusula **SELECT** incluye las columnas de la orden y las columnas de las planchas. La cláusula **FROM** incluye un **LEFT JOIN** con la tabla de planchas. El resultado es una lista de tipos anónimos con las propiedades **NumeroOrden**, **Cliente** y **Planchas**.

Error común. si la propiedad del DTO es `List<T>`, la proyección debe entregar ese tipo de colección; por eso el ejemplo usa `ToList()` dentro de la proyección. Otras formas de resultado pueden modelarse de manera diferente.

Proyectar con funciones de agregación

Se pueden usar funciones de agregación dentro de una proyección para calcular valores como **Count**, **Sum**, **Average**, **Min** y **Max**. EF Core traduce estas funciones a SQL.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        TotalPlanchas = o.Planhas.Count(),
        PesoTotal = o.Planhas.Sum(p => p.Peso)
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden a un tipo anónimo con el total de planchas y el peso total. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], (
    SELECT COUNT(*) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [TotalPlanchas], (
    SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [PesoTotal]
FROM [OrdenesFabricacion] AS [o]
```

SQL

La cláusula **SELECT** incluye dos subconsultas para calcular el total de planchas y el peso total. El resultado es una lista de tipos anónimos con las propiedades **NumeroOrden**, **Cliente**, **TotalPlanchas** y **PesoTotal**.

Error común. si la propiedad del DTO es `List<T>`, la proyección debe entregar ese tipo de colección; por eso el ejemplo usa `ToList()` dentro de la proyección. Otras formas de resultado pueden modelarse de manera diferente.

El proyecto AceriaData

En el proyecto **AceriaData**, se añaden proyecciones al repositorio de órdenes. Las proyecciones se encapsulan en métodos que devuelven DTOs o tipos anónimos. Los DTOs se colocan en la capa de aplicación. La base de datos sigue siendo **AceriaDB** en **SQL Server LocalDB**.

Validación técnica en AceriaData

En el checkpoint **M03/PROYECTO/3.3**, esta materia se ejecuta sobre **SQL Server LocalDB** después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el **Change Tracker** antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `Select`, tipos anónimos, `Distinct`, `OrdenResumenDto`, `OrdenConTotalesDto`, proyección antes de materializar. El punto termina con el marcador **E2E 3.3 OK**.

Resumen de la teoría

- Una proyección transforma cada elemento de una secuencia en una nueva forma.
- `Select` es el operador de proyección.
- Los tipos anónimos se definen en el momento de la proyección.
- Los DTOs son clases con nombre que se usan para transferir datos.
- `Distinct` elimina los duplicados.
- La proyección reduce el volumen de datos transferidos.

- La proyección se combina con Where y OrderBy.
- La proyección de propiedades de navegación genera un JOIN.
- Las funciones de agregación se pueden usar dentro de una proyección.
- En el proyecto AceriaData se añaden proyecciones al repositorio de órdenes.

Punto 3.4 — Proyecciones a DTOs

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en las proyecciones a DTOs en el proyecto AceriaData, incluyendo proyecciones con constructor, proyecciones de colecciones de navegación, proyecciones anidadas y proyecciones con SelectMany.

Objetivos de aprendizaje

- Comprender qué es un DTO y por qué se usa en las proyecciones.
- Proyectar a DTOs con propiedades inicializables.
- Proyectar a DTOs con constructor parametrizado.
- Proyectar colecciones de navegación dentro de un DTO.
- Proyectar DTOs anidados.
- Utilizar SelectMany para aplanar colecciones.
- Analizar el SQL generado por las proyecciones a DTOs.
- Aplicar estas proyecciones al proyecto AceriaData.

Teoría

Qué es un DTO y por qué se usa

Un DTO (Data Transfer Object) es una clase con nombre que se usa para transferir datos entre capas. A diferencia de los tipos anónimos, un DTO tiene un nombre que se puede usar fuera del método donde se creó, se puede validar, se puede documentar y se puede versionar. Los DTOs son especialmente útiles en arquitecturas limpias, donde la capa de aplicación expone DTOs y no entidades de dominio.

```
public class OrdenResumenDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
}
```

CSHARP

La primera línea declara la clase. Las siguientes líneas declaran las propiedades. El DTO representa un resumen de una orden con tres propiedades. Se usa para transferir datos de la orden sin cargar la entidad completa.

Los DTOs aportan varias ventajas. La primera es el desacoplamiento: la capa de aplicación no expone las entidades del dominio, lo que evita que los cambios en el dominio afecten a los consumidores. La segunda es el control: se decide qué propiedades se exponen y cuáles no. La tercera es la seguridad: se evita exponer propiedades sensibles como contraseñas o datos internos. La cuarta es la eficiencia: se proyecta solo lo que se necesita, reduciendo el volumen de datos transferidos.

Proyectar a un DTO con propiedades inicializables

La forma más común de proyectar a un DTO es usar el inicializador de objeto. Se crea una instancia del DTO y se asignan las propiedades en la proyección.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
```

CSHARP

```

    })
    .ToList();

```

La primera línea inicia la consulta. La segunda línea proyecta cada orden a un `OrdenResumenDto` usando el inicializador de objeto. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```

SELECT [o].[NumeroOrden], [o].[Cliente], [o].[Estado]
FROM [OrdenesFabricacion] AS [o]

```

SQL

La cláusula `SELECT` solo incluye las columnas proyectadas. El DTO se materializa con los valores correspondientes. Este patrón es el más habitual y el más sencillo de leer.

Error común. si el DTO tiene una propiedad que no se asigna en la proyección, EF Core deja la propiedad con su valor por defecto. Si el DTO tiene la propiedad marcada como requerida, se debe asignar obligatoriamente.

Proyectar a un DTO con constructor parametrizado

Un DTO puede tener un constructor que acepta los valores de las propiedades. Se proyecta pasando los valores al constructor. Este patrón es útil cuando el DTO tiene propiedades de solo lectura o cuando se quiere garantizar que el DTO se construye con todos los valores.

```

public class OrdenResumenDto
{
    public string NumeroOrden { get; }
    public string Cliente { get; }
    public string Estado { get; }

    public OrdenResumenDto(string numeroOrden, string cliente, string estado)
    {
        NumeroOrden = numeroOrden;
        Cliente = cliente;
        Estado = estado;
    }
}

var resultado = context.OrdenesFabricacion
    .Select(o => new OrdenResumenDto(o.NumeroOrden, o.Cliente, o.Estado))
    .ToList();

```

CSHARP

La primera línea declara el DTO. Las siguientes líneas declaran las propiedades como solo lectura. La sexta línea declara el constructor. Las siguientes líneas asignan los parámetros a las propiedades. La penúltima línea proyecta cada orden al DTO pasando los valores al constructor. La última línea materializa la consulta.

El SQL generado es el mismo que con el inicializador de objeto. La diferencia está en el código de C#: el constructor garantiza que el DTO se construye con todos los valores y que las propiedades son inmutables.

Error común. confundir construcción del DTO con traducción SQL. La proyección superior puede completar en el cliente una parte no traducible después de obtener del servidor los valores necesarios; una expresión no traducible usada antes de esa proyección superior puede provocar una excepción. Mantener el constructor simple hace ese comportamiento más predecible.

Proyectar colecciones de navegación dentro de un DTO

Un DTO puede contener una colección de otros DTOs. Se proyecta la colección de navegación dentro del DTO usando una subproyección con `Select` y `ToList`.

```

public class OrdenConPlanchasDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public List<PlanchaDto> Planchas { get; set; } = new();
}

public class PlanchaDto
{
    public int Id { get; set; }
    public double Espesor { get; set; }
}

```

CSHARP

```

    public decimal Peso { get; set; }
}

var resultado = context.OrdenesFabricacion
    .Select(o => new OrdenConPlanchasDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Planchas = o.Planchas.Select(p => new PlanchaDto
        {
            Id = p.Id,
            Espesor = p.Espesor,
            Peso = p.Peso
        }).ToList()
    })
    .ToList();

```

La primera línea declara el DTO `OrdenConPlanchasDto`. Las siguientes líneas declaran sus propiedades, incluyendo la colección `Planchas` de tipo `List<PlanchaDto>`. La sexta línea declara el DTO `PlanchaDto`. Las siguientes líneas declaran sus propiedades. La penúltima línea proyecta cada orden a un `OrdenConPlanchasDto` con la colección de planchas proyectada. La última línea materializa la consulta.

El SQL generado tiene la siguiente forma:

```

SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Id], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
ORDER BY [o].[Id], [p].[Id]

```

SQL

La cláusula `SELECT` incluye las columnas de la orden y las columnas de las planchas. La cláusula `FROM` incluye un `LEFT JOIN` con la tabla de planchas. EF Core agrupa las filas por orden y construye la colección de planchas.

Error común. convertir `ToList()` dentro de toda proyección de colección en una regla universal. Es necesario cuando el *shape* de destino exige una `List<T>` concreta; otros destinos pueden exponer otra forma de secuencia. La traducción y el tipo final deben comprobarse para el DTO elegido.

Proyectar DTOs anidados

Un DTO puede contener otro DTO como propiedad. Se proyecta el DTO anidado dentro del DTO principal.

```

public class OrdenConDetalleDto
{
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DetalleDto? Detalle { get; set; }
}

public class DetalleDto
{
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
}

var resultado = context.OrdenesFabricacion
    .Select(o => new OrdenConDetalleDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Detalle = o.Detalle == null ? null : new DetalleDto
        {
            ComposicionQuimica = o.Detalle.ComposicionQuimica,
            TemperaturaColada = o.Detalle.TemperaturaColada
        }
    })
    .ToList();

```

CSHARP

La primera línea declara el DTO `OrdenConDetalleDto`. Las siguientes líneas declaran sus propiedades, incluyendo la propiedad `Detalle` de tipo `DetalleDto?`. La sexta línea declara el DTO `DetalleDto`. Las siguientes líneas declaran sus propiedades. La penúltima línea proyecta cada orden a un `OrdenConDetalleDto` con el detalle proyectado. La última línea materializa la consulta.

El operador ternario `o.Detalle == null ? null : new DetalleDto { ... }` comprueba si el detalle existe antes de proyectarlo. Si no existe, la propiedad `Detalle` del DTO queda a `null`.

El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [d].[ComposicionQuimica], [d].[TemperaturaColada]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [DetallesOrden] AS [d] ON [o].[Id] = [d].[OrdenId]
```

SQL

La cláusula SELECT incluye las columnas de la orden y las columnas del detalle. La cláusula FROM incluye un LEFT JOIN con la tabla de detalles. EF Core comprueba si el detalle existe antes de materializarlo.

Error común. si se proyecta el DTO anidado sin comprobar si la entidad relacionada existe, se produce una `NullReferenceException` en tiempo de ejecución. Se debe comprobar si la entidad relacionada es null antes de proyectarla.

El operador SelectMany

El operador `SelectMany` aplanar una secuencia de secuencias en una sola secuencia. Se usa cuando se quiere proyectar una colección de navegación como una lista plana.

```
var resultado = context.OrdenesFabricacion
    .SelectMany(o => o.Planchas, (o, p) => new
    {
        o.NumeroOrden,
        o.Cliente,
        p.Espesor,
        p.Peso
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea usa `SelectMany` para aplanar la colección de planchas. La tercera línea proyecta cada combinación de orden y plancha a un tipo anónimo. La cuarta línea materializa la consulta.

El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
```

SQL

La cláusula SELECT incluye las columnas de la orden y las columnas de las planchas. La cláusula FROM incluye un INNER JOIN con la tabla de planchas. El resultado es una lista plana con una fila por cada combinación de orden y plancha.

Error común. si se usa `SelectMany` con una colección vacía, la orden no aparece en el resultado. Se debe usar `DefaultIfEmpty` si se quieren incluir las órdenes sin planchas.

SelectMany con DefaultIfEmpty

El operador `DefaultIfEmpty` permite incluir las entidades principales aunque la colección de navegación esté vacía. Se combina con `SelectMany` para hacer un LEFT JOIN.

```
var resultado = context.OrdenesFabricacion
    .SelectMany(o => o.Planchas.DefaultIfEmpty(), (o, p) => new
    {
        o.NumeroOrden,
        o.Cliente,
        Espesor = p == null ? 0 : p.Espesor,
        Peso = p == null ? 0 : p.Peso
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea usa `SelectMany` con `DefaultIfEmpty` para incluir las órdenes sin planchas. La tercera línea proyecta cada combinación. La cuarta línea materializa la consulta.

El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
```

SQL

La cláusula FROM incluye un LEFT JOIN con la tabla de planchas. Las órdenes sin planchas aparecen con valores nulos en las columnas de las planchas. La proyección usa el operador ternario para convertir los nulos en ceros.

Error común. si no se comprueba si p es null antes de acceder a sus propiedades, se produce una `NullReferenceException`. Se debe comprobar siempre.

Proyectar con GroupBy y agregaciones

Se puede combinar `GroupBy` con proyecciones para obtener resúmenes agrupados. EF Core traduce al `GROUP BY` relacional los patrones compatibles —especialmente clave de grupo más agregados escalares—; otras formas pueden traducirse de otra manera o componerse después de recuperar filas.

```
var resultado = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Select(g => new
    {
        Cliente = g.Key,
        TotalOrdenes = g.Count(),
        TotalPlanchas = g.Sum(o => o.Planchas.Count())
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea proyecta cada grupo a un tipo anónimo. La cuarta línea materializa la consulta.

El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [TotalOrdenes], COALESCE(SUM((
    SELECT COUNT(*) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
)), 0) AS [TotalPlanchas]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
```

SQL

La cláusula SELECT incluye el cliente, el total de órdenes y el total de planchas. La cláusula GROUP BY agrupa por cliente. El resultado es una lista de tipos anónimos con los totales por cliente.

Error común. en el patrón relacional `GroupBy + agregados`, la proyección debe poder expresarse con la clave del grupo, agregados o expresiones traducibles derivadas de ellos. Otras formas de `GroupBy` pueden tener una traducción diferente o no traducirse como `GROUP BY`; hay que comprobar el *shape* real.

Proyectar con funciones de agregación anidadas

Se pueden usar funciones de agregación anidadas dentro de una proyección. Por ejemplo, contar las planchas de cada orden y sumar los pesos.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        TotalPlanchas = o.Planchas.Count(),
        PesoTotal = o.Planchas.Sum(p => p.Peso),
        PesoPromedio = o.Planchas.Select(p => (decimal?)p.Peso).Average() ?? 0m
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden a un tipo anónimo. La tercera línea incluye el número de orden. La cuarta línea incluye el cliente. La quinta línea calcula el total de planchas. La sexta línea calcula el peso total. La séptima línea calcula el peso promedio. La octava línea materializa la consulta.

El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], (
    SELECT COUNT(*) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [TotalPlanchas], (
    SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [PesoTotal], (
    SELECT COALESCE(AVG([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [PesoPromedio]
FROM [OrdenesFabricacion] AS [o]
```

SQL

La cláusula SELECT incluye tres subconsultas para calcular las agregaciones. El resultado es una lista de tipos anónimos con los totales.

Error común. una agregación sobre una relación vacía debe tener una semántica definida. En este ejemplo se proyecta a `decimal?` y se aplica `?? 0m`, de modo que el contrato devuelve cero sin depender de una afirmación genérica sobre todas las sobrecargas de `Average()`.

Proyectar con navegación a través de SelectMany

Se puede combinar `SelectMany` con proyecciones para obtener datos de entidades relacionadas a través de varias relaciones.

```
var resultado = context.OrdenesFabricacion
    .SelectMany(o => o.Planchas, (o, p) => new { o, p })
    .Select(x => new
    {
        x.o.NumeroOrden,
        x.o.Cliente,
        x.p.Espesor,
        x.p.Peso
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea usa `SelectMany` para combinar órdenes y planchas. La tercera línea proyecta cada combinación a un tipo anónimo. La cuarta línea materializa la consulta.

El SQL generado es similar al de `SelectMany` simple. La diferencia es que aquí se separa en dos pasos: primero se combinan las entidades y después se proyectan.

Error común. si se anida demasiado la proyección, el código se vuelve difícil de leer. Se recomienda mantener las proyecciones simples y combinar varios `Select` si es necesario.

El proyecto AceriaData

En el proyecto `AceriaData`, se añaden DTOs para las proyecciones más habituales: resúmenes de órdenes, órdenes con planchas, órdenes con detalle, órdenes con totales. Los DTOs se colocan en la capa de aplicación. Los métodos de proyección se añaden al repositorio de órdenes. La base de datos sigue siendo `AceriaDB` en `SQL Server LocalDB`.

Validación técnica en AceriaData

En el checkpoint `M03/PROYECTO/3.4`, esta materia se ejecuta sobre `SQL Server LocalDB` después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el `Change Tracker` antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre DTOs con nombre, colecciones de navegación proyectadas, DTOs anidados, `OrdenCompletoDto`, lecturas `AsNoTracking()`. El punto termina con el marcador `E2E 3.4 OK`.

Resumen de la teoría

- Un DTO es una clase con nombre que se usa para transferir datos.
- Los DTOs aportan desacoplamiento, control, seguridad y eficiencia.
- Se proyecta a un DTO con inicializador de objeto o con constructor.
- Se pueden proyectar colecciones de navegación dentro de un DTO.
- Se pueden proyectar DTOs anidados.

- SelectMany aplanar una secuencia de secuencias.
- DefaultIfEmpty permite incluir entidades sin colección relacionada.
- GroupBy se combina con proyecciones para obtener resúmenes agrupados.
- Las funciones de agregación se pueden anidar en una proyección.
- En el proyecto AceriaData se añaden DTOs y métodos de proyección al repositorio.

Punto 3.5 — Consultas de agregación: Count, Sum, Average, Min y Max

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se añaden consultas de agregación al proyecto AceriaData, incluyendo Count, Sum, Average, Min, Max y GroupBy, encapsuladas en métodos del repositorio.

Objetivos de aprendizaje

- Comprender qué es una función de agregación y para qué sirve.
- Diferenciar entre agregaciones simples y agregaciones agrupadas.
- Utilizar Count, LongCount, Any y All.
- Utilizar Sum, Average, Min y Max.
- Combinar GroupBy con funciones de agregación.
- Comprender el impacto de las agregaciones en el SQL generado.
- Evitar el problema de las agregaciones sobre colecciones vacías.
- Aplicar estas consultas al proyecto AceriaData.

Teoría

Qué es una función de agregación

Una función de agregación es una operación que toma una secuencia de valores y devuelve un único valor. En SQL, las funciones de agregación más habituales son COUNT, SUM, AVG, MIN y MAX. En LINQ, estas funciones se expresan con los métodos Count, Sum, Average, Min y Max. EF Core traduce estos métodos a las funciones SQL correspondientes.

```
var total = context.OrdenesFabricacion.Count();
var pesoTotal = context.PlanchasAcero.Sum(p => p.Peso);
var pesoPromedio = context.PlanchasAcero
    .Select(p => (decimal?)p.Peso)
    .Average() ?? 0m;
```

CSHARP

La primera línea cuenta todas las órdenes. La segunda línea suma el peso de todas las planchas. La tercera línea calcula el peso promedio de las planchas. Cada una de estas consultas se traduce a una función de agregación en SQL y devuelve un único valor.

Las funciones de agregación son útiles para obtener resúmenes de datos sin tener que cargar todas las filas en memoria. El cálculo se realiza en el servidor y solo se transfiere el resultado.

El operador Count

El operador Count devuelve el número de elementos de una secuencia. Se traduce a COUNT(*) en SQL. Se puede usar sin argumentos para contar todos los elementos o con una condición para contar los que la cumplen.

```
var totalOrdenes = context.OrdenesFabricacion.Count();
var ordenesPendientes = context.OrdenesFabricacion.Count(o => o.Estado == "Pendiente");
```

CSHARP

La primera línea cuenta todas las órdenes. La segunda línea cuenta las órdenes pendientes. El SQL generado tiene la siguiente forma:

```
SELECT COUNT(*) FROM [OrdenesFabricacion] AS [o];
SELECT COUNT(*) FROM [OrdenesFabricacion] AS [o] WHERE [o].[Estado] = N'Pendiente';
```

SQL

La primera consulta cuenta todas las filas. La segunda consulta cuenta solo las filas que cumplen la condición. En ambos casos, el resultado es un número entero.

Error común. `var lista = consulta.ToList(); var total = lista.Count;` ejecuta una sola consulta de base de datos y cuenta después en memoria. Si solo se necesita el número de filas, `consulta.Count()` evita transferirlas. Solo habría una segunda consulta si, después de `ToList()`, se volviera a ejecutar por separado `consulta.Count()` sobre el `IQueryable` original.

El operador LongCount

El operador `LongCount` es similar a `Count`, pero devuelve un `long` en lugar de un `int`. Se usa cuando el número de elementos puede superar el rango de `int` (más de dos mil millones).

```
var total = context.OrdenesFabricacion.LongCount();
```

CSHARP

La primera línea cuenta todas las órdenes y devuelve un `long`. El SQL generado es el mismo que el de `Count`.

Error común. si se usa `Count` en una tabla con más de dos mil millones de filas, se produce un desbordamiento. Se debe usar `LongCount` en esos casos.

El operador Any

El operador `Any` devuelve `true` si la secuencia contiene al menos un elemento. Se traduce a `EXISTS` en SQL, que es más eficiente que `COUNT(*) > 0` porque se detiene en el primer elemento encontrado.

```
var hayOrdenes = context.OrdenesFabricacion.Any();
var hayPendientes = context.OrdenesFabricacion.Any(o => o.Estado == "Pendiente");
```

CSHARP

La primera línea comprueba si hay alguna orden. La segunda línea comprueba si hay alguna orden pendiente. El SQL generado tiene la siguiente forma:

```
SELECT CASE WHEN EXISTS (SELECT 1 FROM [OrdenesFabricacion] AS [o]) THEN 1 ELSE 0 END;
SELECT CASE WHEN EXISTS (SELECT 1 FROM [OrdenesFabricacion] AS [o] WHERE [o].[Estado] = N'Pendiente') THEN 1 ELSE 0 END;
```

SQL

La primera consulta comprueba si existe al menos una fila. La segunda consulta comprueba si existe al menos una fila que cumple la condición. En ambos casos, el resultado es un booleano.

Error común. si solo interesa saber si existe alguna fila, `Any()` expresa mejor la intención y EF Core lo traduce a una comprobación `EXISTS`; `Count() > 0` solicita calcular un recuento que no se necesita.

El operador All

El operador `All` devuelve `true` si todos los elementos de la secuencia cumplen una condición. Se traduce a `NOT EXISTS` en SQL.

```
var todasPendientes = context.OrdenesFabricacion.All(o => o.Estado == "Pendiente");
```

CSHARP

La primera línea comprueba si todas las órdenes están pendientes. El SQL generado tiene la siguiente forma:

```
SELECT CASE WHEN NOT EXISTS (
    SELECT 1 FROM [OrdenesFabricacion] AS [o] WHERE [o].[Estado] <> N'Pendiente'
) THEN 1 ELSE 0 END;
```

SQL

La consulta comprueba si no existe ninguna fila que no cumpla la condición. Si no existe ninguna, todas cumplen la condición.

Error común. si la secuencia está vacía, All devuelve true porque no hay ningún elemento que no cumpla la condición. Es un comportamiento correcto según la lógica matemática, pero puede sorprender.

El operador Sum

El operador Sum suma los valores de una secuencia. Se traduce a SUM en SQL. Se puede usar sobre una propiedad numérica o sobre una proyección.

```
var pesoTotal = context.PlanchasAcero.Sum(p => p.Peso);
var pesoTotalPorOrden = context.OrdenesFabricacion
    .Where(o => o.Id == 1)
    .SelectMany(o => o.Planchas)
    .Sum(p => p.Peso);
```

CSHARP

La primera línea suma el peso de todas las planchas. La segunda línea suma el peso de las planchas de una orden concreta. El SQL generado tiene la siguiente forma:

```
SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p];
SELECT COALESCE(SUM([p].[Peso]), 0.0)
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[Id] = 1;
```

SQL

La primera consulta suma el peso de todas las planchas. La segunda consulta suma el peso de las planchas de la orden 1. EF Core usa COALESCE para devolver cero cuando no hay filas.

Error común. no conviene generalizar el resultado de una agregación vacía sin considerar el operador, el tipo y el proveedor. En AceriaData se hace explícito el contrato proyectando el peso como decimal? y aplicando ?? 0m cuando corresponde.

El operador Average

El operador Average calcula el promedio de los valores de una secuencia. Se traduce a AVG en SQL.

```
var pesoPromedio = context.PlanchasAcero
    .Select(p => (decimal?)p.Peso)
    .Average() ?? 0m;
```

CSHARP

La primera línea calcula el peso promedio de las planchas. El SQL generado tiene la siguiente forma:

```
SELECT AVG([p].[Peso]) FROM [PlanchasAcero] AS [p];
```

SQL

La consulta SQL calcula el promedio. Al proyectar a decimal?, un conjunto sin valores puede representarse como null; el ?? 0m de AceriaData fija después el valor de retorno del método.

Error común. las sobrecargas no anulables de Average() pueden fallar sobre una secuencia vacía, mientras que las anulables pueden devolver null. AceriaData proyecta a decimal? y aplica ?? 0m para que el comportamiento esté definido.

El operador Min

El operador Min devuelve el valor mínimo de una secuencia. Se traduce a MIN en SQL.

```
var pesoMinimo = context.PlanchasAcero.Min(p => p.Peso);
var fechaMasAntigua = context.OrdenesFabricacion.Min(o => o.FechaCreacion);
```

CSHARP

La primera línea obtiene el peso mínimo de las planchas. La segunda línea obtiene la fecha más antigua de las órdenes. El SQL generado tiene la siguiente forma:

```
SELECT MIN([p].[Peso]) FROM [PlanchasAcero] AS [p];
SELECT MIN([o].[FechaCreacion]) FROM [OrdenesFabricacion] AS [o];
```

SQL

La primera consulta devuelve el peso mínimo. La segunda consulta devuelve la fecha mínima.

Error común. `Min()` sobre una secuencia vacía de un tipo de valor no anulable no devuelve automáticamente el valor por defecto; puede fallar. AceriaData proyecta a `decimal?` y aplica `?? 0m` cuando desea definir explícitamente un resultado para el conjunto vacío.

El operador Max

El operador `Max` devuelve el valor máximo de una secuencia. Se traduce a `MAX` en SQL.

```
var pesoMaximo = context.PlanchasAcero.Max(p => p.Peso);
var fechaMasReciente = context.OrdenesFabricacion.Max(o => o.FechaCreacion);
```

CSHARP

La primera línea obtiene el peso máximo de las planchas. La segunda línea obtiene la fecha más reciente de las órdenes. El SQL generado tiene la siguiente forma:

```
SELECT MAX([p].[Peso]) FROM [PlanchasAcero] AS [p];
SELECT MAX([o].[FechaCreacion]) FROM [OrdenesFabricacion] AS [o];
```

SQL

La primera consulta devuelve el peso máximo. La segunda consulta devuelve la fecha máxima.

Error común. `Max()` sobre una secuencia vacía de un tipo de valor no anulable no devuelve automáticamente el valor por defecto; puede fallar. AceriaData proyecta a `decimal?` y aplica `?? 0m` cuando desea definir explícitamente un resultado para el conjunto vacío.

El operador GroupBy

El operador `GroupBy` agrupa los elementos de una secuencia por una clave. Se traduce a `GROUP BY` en SQL. Se combina con funciones de agregación para obtener resúmenes por grupo.

```
var resumenPorCliente = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Select(g => new
    {
        Cliente = g.Key,
        TotalOrdenes = g.Count(),
        FechaMasReciente = g.Max(o => o.FechaCreacion)
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea proyecta cada grupo. La cuarta línea incluye el cliente. La quinta línea cuenta las órdenes del grupo. La sexta línea obtiene la fecha más reciente del grupo. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [TotalOrdenes], MAX([o].[FechaCreacion]) AS [FechaMasReciente]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
```

SQL

La cláusula `SELECT` incluye la clave del grupo, el conteo y el máximo. La cláusula `GROUP BY` agrupa por cliente. El resultado es una lista de tipos anónimos con los resúmenes por cliente.

Error común. en el patrón relacional `GroupBy + agregados`, la proyección debe poder expresarse con la clave del grupo, agregados o expresiones traducibles derivadas de ellos. Otras formas de `GroupBy` pueden tener una traducción diferente o no traducirse como `GROUP BY`; hay que comprobar el *shape* real.

El operador GroupBy con varias claves

El operador GroupBy acepta varias claves usando un tipo anónimo. Se traduce a GROUP BY con varias columnas.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => new { o.Cliente, o.Estado })
    .Select(g => new
    {
        Cliente = g.Key.Cliente,
        Estado = g.Key.Estado,
        Total = g.Count()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente y estado. La tercera línea proyecta cada grupo. La cuarta línea incluye el cliente. La quinta línea incluye el estado. La sexta línea cuenta las órdenes del grupo. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], [o].[Estado], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente], [o].[Estado]
```

SQL

La cláusula SELECT incluye las dos claves del grupo y el conteo. La cláusula GROUP BY agrupa por cliente y estado. El resultado es una lista de tipos anónimos con los resúmenes por cliente y estado.

Error común. si se agrupa por una propiedad anulable, las filas con valor null se agrupan en un grupo separado. Se debe tener en cuenta al interpretar los resultados.

El operador GroupBy con filtro Having

El filtro Having se aplica a los grupos después de la agrupación. En LINQ, se expresa con un Where después del GroupBy.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Where(g => g.Count() > 1)
    .Select(g => new
    {
        Cliente = g.Key,
        Total = g.Count()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea filtra los grupos con más de una orden. La cuarta línea proyecta cada grupo. La quinta línea incluye el cliente. La sexta línea cuenta las órdenes. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
HAVING COUNT(*) > 1
```

SQL

La cláusula GROUP BY agrupa por cliente. La cláusula HAVING filtra los grupos con más de una orden. El resultado es una lista de tipos anónimos con los clientes que tienen más de una orden.

Error común. si se usa Where antes del GroupBy, el filtro se aplica a las filas antes de agrupar. Si se usa Where después del GroupBy, el filtro se aplica a los grupos y se traduce a HAVING. El orden de las llamadas importa.

Agregaciones anidadas

Se pueden usar funciones de agregación anidadas dentro de una proyección. Por ejemplo, contar las planchas de cada orden y sumar los pesos.

CSHARP

```
var resumen = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        TotalPlanchas = o.Planchas.Count(),
        PesoTotal = o.Planchas.Sum(p => p.Peso),
        PesoPromedio = o.Planchas.Select(p => (decimal?)p.Peso).Average() ?? 0m
    })
    .ToList();
```

La primera línea inicia la consulta. La segunda línea proyecta cada orden. La tercera línea incluye el número de orden. La cuarta línea incluye el cliente. La quinta línea cuenta las planchas. La sexta línea suma los pesos. La séptima línea calcula el promedio. La octava línea materializa la consulta. El SQL generado tiene la siguiente forma:

SQL

```
SELECT [o].[NumeroOrden], [o].[Cliente], (
    SELECT COUNT(*) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [TotalPlanchas], (
    SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [PesoTotal], (
    SELECT COALESCE(AVG([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
) AS [PesoPromedio]
FROM [OrdenesFabricacion] AS [o]
```

La cláusula SELECT incluye tres subconsultas para calcular las agregaciones. El resultado es una lista de tipos anónimos con los totales.

Error común. una agregación sobre una relación vacía debe tener una semántica definida. En este ejemplo se proyecta a `decimal?` y se aplica `?? 0m`, de modo que el contrato devuelve cero de forma explícita.

Agregaciones sobre colecciones vacías

En LINQ to Objects, una secuencia vacía devuelve cero con `Count()` y con las sobrecargas numéricas habituales de `Sum()`, mientras que las sobrecargas no anulables de `Average()`, `Min()` y `Max()` pueden lanzar `InvalidOperationException`; las variantes anulables pueden devolver `null`. Sobre `IQueryable`, además intervienen la semántica SQL y la traducción del proveedor. `AceriaData` hace explícito el resultado de agregados sensibles al vacío proyectando a nullable y aplicando `?? 0m`.

CSHARP

```
var ordenes = context.OrdenesFabricacion.Where(o => o.Id == 999).ToList();

var total = ordenes.Count(); // 0
var suma = ordenes.Sum(o => o.Id); // 0
var promedio = ordenes.Average(o => (double)o.Id); // InvalidOperationException
var minimo = ordenes.Min(o => o.Id); // InvalidOperationException
var maximo = ordenes.Max(o => o.Id); // InvalidOperationException
```

La primera línea obtiene una lista vacía. La segunda línea cuenta los elementos, que es cero. La tercera línea suma los elementos, que es cero. La cuarta línea calcula el promedio y lanza una excepción. La quinta línea obtiene el mínimo y lanza una excepción. La sexta línea obtiene el máximo y lanza una excepción.

Error común. asumir una única semántica para los agregados vacíos. Se debe elegir conscientemente una sobrecarga anulable, comprobar existencia o definir un valor por defecto; en `AceriaData` se usa el patrón nullable + `??` para los agregados de peso.

Agregaciones con DefaultIfEmpty

`DefaultIfEmpty` introduce un elemento por defecto cuando una secuencia está vacía y puede combinarse con agregados en LINQ. En consultas `IQueryable` su traducción depende del patrón y del proveedor; `AceriaData` valida como estrategia principal la proyección a un tipo anulable y la coalescencia explícita del resultado.

CSHARP

```
var promedio = context.PlanchasAcero
    .Where(p => p.OrdenId == 999)
    .Select(p => (double)p.Peso)
    .DefaultIfEmpty(0)
    .Average();
```

La primera línea inicia la consulta. La segunda línea filtra por orden. La tercera línea proyecta el peso. La cuarta línea aplica `DefaultIfEmpty` con valor cero. La quinta línea calcula el promedio. Si no hay planchas, el promedio es cero.

Error común. si se aplica `DefaultIfEmpty` con un valor distinto de cero, el promedio se calcula con ese valor. Se debe usar cero para que no afecte al promedio.

El proyecto AceriaData

En el proyecto `AceriaData`, se añaden métodos de agregación al repositorio de órdenes. Los métodos devuelven valores simples o DTOs con resúmenes agrupados. La capa de aplicación usa estos métodos a través de la unidad de trabajo. La base de datos sigue siendo `AceriaDB` en `SQL Server LocalDB`.

Validación técnica en AceriaData

En el checkpoint `M03/PROYECTO/3.5`, esta materia se ejecuta sobre `SQL Server LocalDB` después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el `Change Tracker` antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `Count / Any / All`, `Sum / Average / Min / Max`, `GroupBy`, agregaciones anulables para colecciones vacías. El punto termina con el marcador `E2E 3.5 OK`.

Resumen de la teoría

- Una función de agregación toma una secuencia y devuelve un único valor.
- `Count` cuenta los elementos. `LongCount` devuelve un `long`.
- `Any` comprueba si hay al menos un elemento. `All` comprueba si todos cumplen una condición.
- `Sum` suma los valores. `Average` calcula el promedio.
- `Min` y `Max` devuelven el mínimo y el máximo.
- `GroupBy` agrupa los elementos por una clave.
- `GroupBy` con varias claves usa un tipo anónimo.
- El filtro `Having` se expresa con un `Where` después del `GroupBy`.
- Las agregaciones anidadas se usan dentro de una proyección.
- Las agregaciones sobre colecciones vacías pueden lanzar excepción.
- `DefaultIfEmpty` permite aplicar agregaciones sobre colecciones vacías.
- En el proyecto `AceriaData` se añaden métodos de agregación al repositorio.

Punto 3.6 — Agrupaciones con proyección

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en `Entity Framework Core`.

Proyecto: Se profundiza en las agrupaciones con proyección en el proyecto `AceriaData`, incluyendo agrupaciones por múltiples claves, agrupaciones con filtros `Having`, agrupaciones con proyecciones anidadas y agrupaciones con `SelectMany`.

Objetivos de aprendizaje

- Comprender qué es una agrupación y cómo se proyecta.
- Agrupar por una clave simple.
- Agrupar por múltiples claves con tipos anónimos.
- Aplicar filtros `Having` después de la agrupación.
- Proyectar grupos con colecciones internas.
- Combinar `GroupBy` con `SelectMany`.
- Analizar el SQL generado por agrupaciones con proyección.
- Aplicar estas agrupaciones al proyecto `AceriaData`.

Teoría

Qué es una agrupación con proyección

Una agrupación con proyección es una operación que agrupa los elementos de una secuencia por una clave y después proyecta cada grupo a una nueva forma. La proyección puede incluir la clave del grupo, funciones de agregación y colecciones internas de los elementos del grupo.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Select(g => new
    {
        Cliente = g.Key,
        TotalOrdenes = g.Count(),
        Ordenes = g.Select(o => o.NumeroOrden).ToList()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea proyecta cada grupo. La cuarta línea incluye la clave del grupo. La quinta línea cuenta las órdenes del grupo. La sexta línea incluye la lista de números de orden del grupo. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [TotalOrdenes]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
ORDER BY [o].[Cliente]
```

SQL

La cláusula SELECT incluye la clave del grupo y el conteo. La cláusula GROUP BY agrupa por cliente. Las proyecciones que combinan GroupBy con colecciones internas tienen traducciones dependientes de la forma exacta y del proveedor. En AceriaData no se presupone una traducción concreta: el checkpoint 3.6 usa dos consultas acotadas — cabeceras agrupadas y filas proyectadas— y compone la colección interna en memoria de forma explícita.

Error común. afirmar que una colección interna dentro de GroupBy implica necesariamente una consulta de grupos seguida de cargas por grupo. La traducción depende del *shape* y del proveedor. La versión validada de AceriaData usa explícitamente dos consultas acotadas y compone después el resultado, por lo que el número de consultas no depende del número de grupos.

Agrupar por una clave simple

La agrupación por una clave simple se realiza con GroupBy y una expresión lambda que devuelve la clave. La clave puede ser una propiedad de la entidad o una expresión calculada.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Select(g => new
    {
        Cliente = g.Key,
        Total = g.Count()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea proyecta cada grupo. La cuarta línea incluye el cliente. La quinta línea cuenta las órdenes. La sexta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
```

SQL

La cláusula SELECT incluye la clave del grupo y el conteo. La cláusula GROUP BY agrupa por cliente. El resultado es una lista de tipos anónimos con el cliente y el total de órdenes.

Error común. en el patrón relacional `GroupBy` + agregados, la proyección debe poder expresarse con la clave del grupo, agregados o expresiones traducibles derivadas de ellos. Otras formas de `GroupBy` pueden tener una traducción diferente o no traducirse como `GROUP BY`; hay que comprobar el *shape* real.

Agrupar por múltiples claves

La agrupación por múltiples claves se realiza con `GroupBy` y una expresión lambda que devuelve un tipo anónimo con las claves. El tipo anónimo se traduce a una cláusula `GROUP BY` con varias columnas.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => new { o.Cliente, o.Estado })
    .Select(g => new
    {
        Cliente = g.Key.Cliente,
        Estado = g.Key.Estado,
        Total = g.Count()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente y estado. La tercera línea proyecta cada grupo. La cuarta línea incluye el cliente. La quinta línea incluye el estado. La sexta línea cuenta las órdenes. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], [o].[Estado], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente], [o].[Estado]
```

SQL

La cláusula `SELECT` incluye las dos claves del grupo y el conteo. La cláusula `GROUP BY` agrupa por cliente y estado. El resultado es una lista de tipos anónimos con el cliente, el estado y el total de órdenes.

Error común. si se agrupa por una propiedad anulable, las filas con valor null se agrupan en un grupo separado. Se debe tener en cuenta al interpretar los resultados.

Agrupar por una expresión calculada

La agrupación por una expresión calculada se expresa con `GroupBy` y una lambda que devuelve el valor calculado. Si el proveedor puede traducir esa expresión, la operación puede ejecutarse en SQL; no debe suponerse que cualquier cálculo tiene traducción automática.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })
    .Select(g => new
    {
        Anio = g.Key.Year,
        Mes = g.Key.Month,
        Total = g.Count()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por año y mes. La tercera línea proyecta cada grupo. La cuarta línea incluye el año. La quinta línea incluye el mes. La sexta línea cuenta las órdenes. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT DATEPART(year, [o].[FechaCreacion]) AS [Anio], DATEPART(month, [o].[FechaCreacion]) AS [Mes], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY DATEPART(year, [o].[FechaCreacion]), DATEPART(month, [o].[FechaCreacion])
```

SQL

La cláusula `SELECT` incluye las funciones `DATEPART` para extraer el año y el mes. La cláusula `GROUP BY` agrupa por las mismas funciones. El resultado es una lista de tipos anónimos con el año, el mes y el total de órdenes.

Error común. si una expresión no puede traducirse en una parte de la consulta que debe ejecutarse en el servidor, EF Core 8 normalmente lanza una excepción; no hace evaluación cliente implícita del filtro. Para continuar en memoria hay que cambiar explícitamente a LINQ to Objects con `AsEnumerable()` o materializar con `ToList()`, asumiendo el coste.

Aplicar filtros Having

El filtro Having se aplica a los grupos después de la agrupación. En LINQ, se expresa con un Where después del GroupBy. El filtro se traduce a una cláusula HAVING en SQL.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Where(g => g.Count() > 1)
    .Select(g => new
    {
        Cliente = g.Key,
        Total = g.Count()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea filtra los grupos con más de una orden. La cuarta línea proyecta cada grupo. La quinta línea incluye el cliente. La sexta línea cuenta las órdenes. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
HAVING COUNT(*) > 1
```

SQL

La cláusula GROUP BY agrupa por cliente. La cláusula HAVING filtra los grupos con más de una orden. El resultado es una lista de tipos anónimos con los clientes que tienen más de una orden.

Error común. si se usa Where antes del GroupBy, el filtro se aplica a las filas antes de agrupar. Si se usa Where después del GroupBy, el filtro se aplica a los grupos y se traduce a HAVING. El orden de las llamadas importa.

Proyectar grupos con colecciones internas

LINQ permite expresar una forma jerárquica que contiene la clave del grupo, agregados y una colección interna. Sin embargo, a diferencia del patrón GroupBy + agregado escalar, la traducción de una colección interna depende de la forma exacta de la consulta y del proveedor; no se debe deducir un SQL concreto ni un patrón N+1 solo por ver ToList() dentro de la proyección.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Select(g => new
    {
        Cliente = g.Key,
        Total = g.Count(),
        Ordenes = g.Select(o => new
        {
            o.NumeroOrden,
            o.Estado,
            o.FechaCreacion
        }).ToList()
    })
    .ToList();
```

CSHARP

El fragmento anterior expresa el *shape* deseado, pero AceriaData no lo usa como prueba de que esa forma completa se traduzca a una única sentencia SQL. El checkpoint 3.6 usa una estrategia explícita y verificable: una consulta obtiene las cabeceras agrupadas y una segunda consulta obtiene las filas proyectadas necesarias; después se compone la colección interna en memoria. El número de consultas queda acotado y no crece con el número de grupos.

SQL comprobable en AceriaData

SQL

```
SELECT [o].[Cliente], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
```

Este SQL representa la parte agregada de cabeceras que sí tiene traducción relacional directa. La colección de órdenes se construye después con la segunda consulta acotada del repositorio.

Error común. describir automáticamente cualquier colección interna dentro de un `GroupBy` como `N+1`. Hay que inspeccionar la traducción real o hacer explícita la frontera cliente/servidor. La versión validada de AceriaData usa siempre un número fijo de consultas para esta demostración.

Combinar GroupBy con SelectMany

La combinación de `GroupBy` con `SelectMany` permite aplanar los grupos y proyectar los elementos individuales. Se usa cuando se quiere una lista plana de elementos agrupados.

CSHARP

```
var resultado = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .SelectMany(g => g, (g, o) => new
    {
        Cliente = g.Key,
        o.NumeroOrden,
        o.Estado
    })
    .ToList();
```

El fragmento muestra la semántica LINQ de aplanar grupos, pero no se presenta como una traducción SQL garantizada: las formas complejas posteriores a `GroupBy` dependen del patrón y del proveedor. Si el objetivo final es una lista plana con la clave repetida, una proyección directa ordenada por cliente suele expresar mejor la consulta relacional. En AceriaData, la evidencia SQL de 3.6 se concentra en los patrones `GroupBy` con agregados y `HAVING` que el checkpoint ejecuta y valida.

Error común. si se combina `GroupBy` con `SelectMany` sin una proyección clara, el resultado puede ser confuso. Se debe usar cuando se quiere una lista plana con la clave del grupo repetida en cada fila.

Agrupaciones con proyecciones anidadas

Se pueden anidar agrupaciones dentro de agrupaciones. Por ejemplo, agrupar por cliente y dentro de cada cliente agrupar por estado.

CSHARP

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Select(g => new
    {
        Cliente = g.Key,
        Total = g.Count(),
        Estados = g.GroupBy(o => o.Estado).Select(eg => new
        {
            Estado = eg.Key,
            Total = eg.Count()
        }).ToList()
    })
    .ToList();
```

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea proyecta cada grupo. La cuarta línea incluye el cliente. La quinta línea cuenta las órdenes. La sexta línea agrupa las órdenes del grupo por estado. La séptima línea proyecta cada subgrupo. La octava línea incluye el estado. La novena línea cuenta las órdenes del subgrupo. La décima línea materializa la colección. La undécima línea materializa la consulta.

El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [Total]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
```

SQL

La cláusula SELECT incluye la clave del grupo y el conteo. Las agrupaciones anidadas pueden traducirse de formas distintas según su shape. En AceriaData se evita asumir una secuencia de consultas por grupo: cuando interesa una estructura jerárquica, se proyectan primero los datos necesarios y la composición final se hace de forma explícita.

Error común. no se debe asumir que una agrupación anidada tendrá una traducción o un número de consultas determinado. Hay que revisar la traducción y, si el *shape* no es adecuado para el servidor, proyectar los datos mínimos necesarios y hacer explícita la composición en memoria.

Agrupaciones con agregaciones múltiples

Se pueden incluir varias funciones de agregación en la proyección de un grupo. Cada función se traduce a una función SQL.

```
var resumen = context.OrdenesFabricacion
    .GroupBy(o => o.Cliente)
    .Select(g => new
    {
        Cliente = g.Key,
        TotalOrdenes = g.Count(),
        FechaMasAntigua = g.Min(o => o.FechaCreacion),
        FechaMasReciente = g.Max(o => o.FechaCreacion),
        TotalPlanchas = g.Sum(o => o.Planchas.Count()),
        PesoTotal = g.Sum(o => o.Planchas.Sum(p => p.Peso))
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea agrupa por cliente. La tercera línea proyecta cada grupo. La cuarta línea incluye el cliente. La quinta línea cuenta las órdenes. La sexta línea obtiene la fecha más antigua. La séptima línea obtiene la fecha más reciente. La octava línea suma las planchas. La novena línea suma el peso. La décima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Cliente], COUNT(*) AS [TotalOrdenes], MIN([o].[FechaCreacion]) AS [FechaMasAntigua], MAX([o].[FechaCreacion]) AS [FechaMasReciente], COALESCE(SUM(
    SELECT COUNT(*) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
), 0) AS [TotalPlanchas], COALESCE(SUM(
    SELECT COALESCE(SUM([p].[Peso]), 0.0) FROM [PlanchasAcero] AS [p] WHERE [o].[Id] = [p].[OrdenId]
), 0.0) AS [PesoTotal]
FROM [OrdenesFabricacion] AS [o]
GROUP BY [o].[Cliente]
```

SQL

La cláusula SELECT incluye la clave del grupo y varias funciones de agregación. La cláusula GROUP BY agrupa por cliente. El resultado es una lista de tipos anónimos con el cliente y los totales.

Error común. si se incluyen muchas funciones de agregación, la consulta puede ser lenta. Se debe revisar el plan de ejecución y considerar índices o consultas separadas.

El proyecto AceriaData

En el proyecto AceriaData, se añaden agrupaciones con proyección al repositorio de órdenes. Las agrupaciones se encapsulan en métodos que devuelven DTOs con resúmenes agrupados. La capa de aplicación usa estos métodos a través de la unidad de trabajo. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Validación técnica en AceriaData

En el checkpoint M03/PROYECTO/3.6, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar Database.Migrate(). La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `GroupBy` por una y varias claves, `HAVING` mediante `Where` posterior al grupo, `resumen + detalle` con dos consultas acotadas, agrupación de peso sobre orden-plancha, SQL real con `ToQueryString()`. El punto termina con el marcador E2E 3.6 OK.

Resumen de la teoría

- Una agrupación con proyección agrupa los elementos y proyecta cada grupo.
- Se agrupa por una clave simple con `GroupBy`.
- Se agrupa por múltiples claves con un tipo anónimo.
- Se agrupa por una expresión calculada con `GroupBy`.
- El filtro `Having` se expresa con un `Where` después del `GroupBy`.
- Se pueden proyectar colecciones internas dentro de un grupo.
- `SelectMany` aplanar los grupos en una lista plana.
- Se pueden anidar agrupaciones dentro de agrupaciones.
- Se pueden incluir varias funciones de agregación en la proyección.
- En el proyecto `AceriaData` se añaden agrupaciones con proyección al repositorio.

Punto 3.7 — Joins y navegación en consultas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se añaden consultas con joins y navegación al proyecto `AceriaData`, incluyendo `Join`, `GroupJoin`, navegación por propiedades y proyecciones combinadas de varias entidades.

Objetivos de aprendizaje

- Comprender qué es un join y cuándo usarlo.
- Diferenciar entre `Join` explícito y navegación por propiedades.
- Comprender qué es un `GroupJoin` y cuándo usarlo.
- Combinar `Join` con `GroupBy` y proyecciones.
- Analizar el SQL generado por los joins.
- Comparar el rendimiento de `Join` explícito versus navegación por propiedades.
- Aplicar joins y navegación al proyecto `AceriaData`.

Teoría

Qué es un join

Un join es una operación que combina filas de dos o más tablas basándose en una condición de coincidencia. En SQL, el join se expresa con la cláusula `JOIN`. En LINQ, el join se expresa con el operador `Join` o con la navegación por propiedades de navegación.

```
var resultado = context.OrdenesFabricacion
    .Join(
        context.PlanchasAcero,
        o => o.Id,
        p => p.OrdenId,
        (o, p) => new { o.NumeroOrden, o.Cliente, p.Espesor, p.Peso })
    .ToList();
```

CSHARP

La primera línea inicia la consulta sobre `OrdenesFabricacion`. La segunda línea invoca el operador `Join`. La tercera línea especifica la clave del lado izquierdo. La cuarta línea especifica la clave del lado derecho. La quinta línea proyecta la combinación. La sexta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
```

SQL

La cláusula SELECT incluye las columnas de ambas tablas. La cláusula FROM indica la tabla principal. La cláusula INNER JOIN combina con la tabla de planchas. El resultado es una lista de tipos anónimos con los datos combinados.

El operador Join en LINQ se traduce a INNER JOIN en SQL. Solo devuelve las filas que tienen coincidencia en ambas tablas. Si una orden no tiene planchas, no aparece en el resultado. Si una plancha no tiene orden, no aparece en el resultado.

La navegación por propiedades

La navegación por propiedades es la forma más natural de expresar un join en EF Core. Se accede a las propiedades de navegación de las entidades y EF Core genera el join automáticamente.

```
var resultado = context.OrdenesFabricacion
    .SelectMany(o => o.Planchas, (o, p) => new { o.NumeroOrden, o.Cliente, p.Espesor, p.Peso })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea usa SelectMany sobre la propiedad de navegación Planchas. La tercera línea proyecta la combinación. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
```

SQL

La cláusula SELECT incluye las columnas de ambas tablas. La cláusula FROM indica la tabla principal. La cláusula INNER JOIN combina con la tabla de planchas. El resultado es el mismo que con Join explícito, pero el código es más legible.

Error común. en el patrón directo que aplanar una navegación de colección correlacionada, SelectMany suele producir semántica de INNER JOIN; añadir DefaultIfEmpty() permite expresar la variante izquierda. Otros *shapes* de SelectMany pueden traducirse a CROSS JOIN, APPLY u otras formas.

La navegación por propiedades con DefaultIfEmpty

El operador DefaultIfEmpty convierte el INNER JOIN en un LEFT JOIN. Se usa cuando se quieren incluir las entidades principales aunque no tengan entidades relacionadas.

```
var resultado = context.OrdenesFabricacion
    .SelectMany(o => o.Planchas.DefaultIfEmpty(), (o, p) => new
    {
        o.NumeroOrden,
        o.Cliente,
        Espesor = p == null ? 0 : p.Espesor,
        Peso = p == null ? 0 : p.Peso
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea usa SelectMany con DefaultIfEmpty para incluir las órdenes sin planchas. La tercera línea proyecta la combinación. La cuarta línea comprueba si la plancha es nula. La quinta línea asigna el espesor. La sexta línea asigna el peso. La séptima línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
```

SQL

La cláusula FROM incluye un LEFT JOIN con la tabla de planchas. Las órdenes sin planchas aparecen con valores nulos en las columnas de las planchas. La proyección usa el operador ternario para convertir los nulos en ceros.

Error común. si no se comprueba si `p` es null antes de acceder a sus propiedades, se produce una `NullReferenceException`. Se debe comprobar siempre.

El operador GroupJoin

El operador `GroupJoin` asocia a cada elemento exterior un grupo de coincidencias interiores. Una proyección que devuelve directamente ese grupo no tiene por qué traducirse a SQL en un proveedor relacional. En EF Core, el patrón reconocido para un `LEFT JOIN` combina `GroupJoin` con un aplanado inmediato mediante `DefaultIfEmpty` (la sintaxis de consulta genera el `SelectMany` correspondiente).

AceriaData usa exactamente ese patrón en `ObtenerLeftJoinOrdenesPlanchas()`:

```
var resultado =
    (from o in context.OrdenesFabricacion
     join p in context.PlanchasAcero on o.Id equals p.OrdenId into planchas
     from p in planchas.DefaultIfEmpty()
     orderby o.NumeroOrden
     select new
     {
         o.NumeroOrden,
         o.Cliente,
         Espesor = p == null ? (decimal?)null : p.Espesor,
         Peso = p == null ? (decimal?)null : p.Peso
     })
    .ToList();
```

CSHARP

El `GroupJoin` crea la agrupación intermedia y `DefaultIfEmpty()` permite conservar la orden aunque no tenga planchas. Al aplanar inmediatamente ese grupo, EF Core puede reconocer el patrón como `LEFT JOIN`:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
ORDER BY [o].[NumeroOrden]
```

SQL

Error común. tratar cualquier `GroupJoin` como si tuviera traducción relacional garantizada. Para un `LEFT JOIN`, se debe respetar el patrón reconocido —agrupación seguida inmediatamente del aplanado con `DefaultIfEmpty`— o comprobar expresamente la traducción del *shape* utilizado.

El operador Join con múltiples condiciones

El operador `Join` acepta una condición de coincidencia simple, pero se puede usar una clave compuesta con un tipo anónimo. Se traduce a un `INNER JOIN` con múltiples condiciones.

```
var resultado = context.OrdenesFabricacion
    .Join(
        context.PlanchasAcero,
        o => new { o.Id, o.Estado },
        p => new { Id = p.OrdenId, Estado = "Pendiente" },
        (o, p) => new { o.NumeroOrden, p.Espesor })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea invoca el operador `Join`. La tercera línea especifica la clave compuesta del lado izquierdo. La cuarta línea especifica la clave compuesta del lado derecho. La quinta línea proyecta la combinación. La sexta línea materializa la consulta.

Error común. si las claves compuestas no coinciden en tipos y orden, el compilador lanza un error. Las claves deben ser del mismo tipo y en el mismo orden.

Joins con más de dos tablas

Se pueden encadenar varios `Join` para combinar más de dos tablas. Cada `Join` añade una tabla adicional.

CSHARP

```
var resultado = context.OrdenesFabricacion
    .Join(
        context.PlanchasAcero,
        o => o.Id,
        p => p.OrdenId,
        (o, p) => new { o, p })
    .Join(
        context.DetallesOrden,
        op => op.o.Id,
        d => d.OrdenId,
        (op, d) => new
        {
            op.o.NumeroOrden,
            op.o.Cliente,
            op.p.Espesor,
            d.ComposicionQuimica
        })
    .ToList();
```

La primera línea inicia la consulta. La segunda línea invoca el primer Join con la tabla de planchas. La tercera línea especifica la clave del lado izquierdo. La cuarta línea especifica la clave del lado derecho. La quinta línea proyecta la combinación. La sexta línea invoca el segundo Join con la tabla de detalles. La séptima línea especifica la clave del lado izquierdo. La octava línea especifica la clave del lado derecho. La novena línea proyecta la combinación final. La décima línea materializa la consulta. El SQL generado tiene la siguiente forma:

SQL

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [d].[ComposicionQuimica]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
INNER JOIN [DetallesOrden] AS [d] ON [o].[Id] = [d].[OrdenId]
```

La cláusula FROM incluye la tabla principal. La primera cláusula INNER JOIN combina con la tabla de planchas. La segunda cláusula INNER JOIN combina con la tabla de detalles. El resultado es una lista de tipos anónimos con los datos combinados.

Error común. varios Join sobre relaciones uno-a-muchos pueden multiplicar el número de filas cuando existen varias coincidencias por clave. Eso no es necesariamente un producto cartesiano completo: conviene distinguir multiplicidad de joins de un CROSS JOIN y revisar el SQL y la cardinalidad reales.

Navegación por propiedades con Include

Include expresa Eager Loading de la navegación junto con la operación principal. En modo single-query una colección o referencia suele resolverse mediante JOIN, mientras que AsSplitQuery() puede dividir la carga de colecciones en varias sentencias; por tanto, Include no equivale siempre a un único LEFT JOIN.

CSHARP

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.Detalle)
    .ToList();
```

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye el detalle. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

SQL

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ..., [p].[Id], [p].[Espesor], ..., [d].[Id], [d].[ComposicionQuimica], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [DetallesOrden] AS [d] ON [o].[Id] = [d].[OrdenId]
ORDER BY [o].[Id], [p].[Id]
```

La cláusula SELECT incluye todas las columnas de todas las tablas. La cláusula FROM indica la tabla principal. Las cláusulas LEFT JOIN combinan con las tablas relacionadas. El resultado es una lista de órdenes con sus planchas y su detalle cargados.

Error común. varias colecciones hermanas incluidas al mismo nivel pueden multiplicar filas en una consulta única. AsSplitQuery() es una alternativa para ese shape, pero debe evaluarse frente al coste de ejecutar varias consultas.

Navegación por propiedades con ThenInclude

El operador ThenInclude carga las entidades relacionadas de las entidades incluidas. Se usa después de Include.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(oa => oa.Aleacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de entidades intermedias. La tercera línea incluye la aleación de cada entidad intermedia. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
LEFT JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
ORDER BY [o].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]
```

SQL

La cláusula SELECT incluye todas las columnas de todas las tablas. La cláusula FROM indica la tabla principal. Las cláusulas LEFT JOIN combinan con las tablas relacionadas. El resultado es una lista de órdenes con sus entidades intermedias y sus aleaciones cargadas.

Error común. si se usa ThenInclude sin Include previo, el compilador lanza un error. ThenInclude solo se puede usar después de Include.

Navegación por propiedades con proyección

Las navegaciones se pueden usar dentro de proyecciones para seleccionar solo las columnas necesarias. La forma SQL concreta —JOIN, subconsulta u otra construcción— depende del patrón y del proveedor y debe verificarse con el SQL generado.

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
        o.NumeroOrden,
        o.Cliente,
        Planchas = o.Planchas
            .Select(p => new
            {
                p.Espesor,
                p.Peso
            })
            .ToList()
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea proyecta cada orden. La tercera línea incluye el número de orden. La cuarta línea incluye el cliente. La quinta línea proyecta la colección de planchas. La sexta línea incluye el espesor. La séptima línea incluye el peso. La octava línea materializa la colección. La novena línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[NumeroOrden], [o].[Cliente], [p].[Espesor], [p].[Peso], [p].[Id]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
ORDER BY [o].[Id], [p].[Id]
```

SQL

La cláusula SELECT incluye las columnas de la orden y las columnas de las planchas. La cláusula FROM indica la tabla principal. La cláusula LEFT JOIN combina con la tabla de planchas. El resultado es una lista de tipos anónimos con las planchas proyectadas.

Error común. si el DTO de destino declara una propiedad List<T>, la proyección debe producir una lista compatible; ToList() dentro de la proyección hace explícita esa forma. No es una regla universal para toda proyección de colecciones, sino una exigencia del shape elegido.

Comparación entre Join explícito y navegación

El Join explícito y la navegación por propiedades generan el mismo SQL en la mayoría de los casos. La diferencia está en el código de C#. La navegación por propiedades es más legible y más natural. El Join explícito es útil cuando no hay propiedades de navegación configuradas o cuando se quiere controlar la clave del join.

```
// Join explícito
var resultado1 = context.OrdenesFabricacion
    .Join(
        context.PlanchasAcero,
        o => o.Id,
        p => p.OrdenId,
        (o, p) => new { o.NumeroOrden, p.Espesor })
    .ToList();

// Navegación por propiedades
var resultado2 = context.OrdenesFabricacion
    .SelectMany(o => o.Planchas, (o, p) => new { o.NumeroOrden, p.Espesor })
    .ToList();
```

CSHARP

La primera consulta usa Join explícito. La segunda consulta usa SelectMany sobre la propiedad de navegación. Ambas generan el mismo SQL. La segunda es más legible.

Error común. si se usa Join explícito sin necesidad, el código se vuelve más complejo y más difícil de mantener. Se debe preferir la navegación por propiedades cuando sea posible.

El proyecto AceriaData

En el proyecto AceriaData, se añaden consultas con joins y navegación al repositorio de órdenes. Los joins se encapsulan en métodos que devuelven DTOs con datos combinados de varias entidades. La capa de aplicación usa estos métodos a través de la unidad de trabajo. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Validación técnica en AceriaData

En el checkpoint M03/PROYECTO/3.7, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar Database.Migrate(). La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre Join explícito, LEFT JOIN mediante patrón GroupJoin/DefaultIfEmpty, navegaciones, proyecciones combinadas, SQL real. El punto termina con el marcador E2E 3.7 OK.

Resumen de la teoría

- Un join combina filas de dos o más tablas por una condición de coincidencia.
- El operador Join se traduce a INNER JOIN.
- SelectMany puede traducirse a INNER JOIN, CROSS JOIN, CROSS APPLY u otras formas según cómo el selector se relacione con el origen.
- En el patrón de LEFT JOIN, DefaultIfEmpty() sobre la colección correlacionada permite conservar la fila exterior; en otros patrones puede producir otra forma SQL.
- El operador GroupJoin agrupa los elementos de la segunda secuencia.
- Se pueden encadenar varios join para combinar más de dos tablas.
- Include carga las entidades relacionadas con LEFT JOIN.
- ThenInclude carga las entidades relacionadas de las entidades incluidas.
- La navegación por propiedades se combina con proyecciones.
- La navegación por propiedades es más legible que el join explícito.
- En el proyecto AceriaData se añaden consultas con joins al repositorio.

Punto 3.8 — Eager Loading con Include y ThenInclude

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en la carga Eager (carga anticipada) en el proyecto AceriaData, incluyendo Include, ThenInclude, AsSplitQuery, Filtered Include y AutoInclude.

Objetivos de aprendizaje

- Comprender qué es la carga Eager y cuándo usarla.
- Cargar colecciones de navegación con Include.
- Cargar referencias de navegación con Include.
- Cargar relaciones anidadas con ThenInclude.
- Aplicar filtros a las colecciones incluidas con Filtered Include.
- Dividir consultas con AsSplitQuery.
- Configurar la carga automática con AutoInclude.
- Analizar el SQL generado por las consultas con Include.
- Aplicar la carga Eager al proyecto AceriaData.

Teoría

Qué es la carga Eager

La carga Eager (carga anticipada) solicita las entidades relacionadas junto con la entidad principal. Por defecto `Include` suele resolverse mediante una consulta con joins para esta forma de consulta, pero `AsSplitQuery()` puede dividirla deliberadamente en varias sentencias coordinadas. Se usa cuando se sabe de antemano qué entidades relacionadas se van a necesitar. La carga se realiza con el método `Include` y sus variantes.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ..., [p].[Id], [p].[Espesor], [p].[Peso], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
ORDER BY [o].[Id], [p].[Id]
```

SQL

La cláusula `SELECT` incluye todas las columnas de la orden y todas las columnas de las planchas. La cláusula `FROM` indica la tabla principal. La cláusula `LEFT JOIN` combina con la tabla de planchas. La cláusula `ORDER BY` ordena por el `Id` de la orden y el `Id` de la plancha. El resultado es una lista de órdenes con sus planchas cargadas.

La carga Eager evita accesos Lazy impredecibles cuando las relaciones se conocen de antemano, pero no existe una estrategia universalmente más eficiente. Cuando una consulta única incluye varias colecciones hermanas al mismo nivel, los `JOIN` pueden multiplicar filas; `AsSplitQuery()` cambia ese coste por varias consultas coordinadas.

Include sobre colecciones

El método `Include` acepta una expresión lambda que apunta a una propiedad de navegación de colección. La colección se carga junto con la entidad principal.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea materializa la consulta.

En este ejemplo las dos navegaciones de colección (`Planchas` y `OrdenesAleaciones`) son hermanas del mismo principal. En modo de consulta única, sus `JOIN` pueden producir un producto cruzado entre las filas relacionadas de ambas colecciones y aumentar mucho el número de filas transferidas.

Error común. varias colecciones hermanas incluidas al mismo nivel pueden multiplicar filas en una consulta única. `AsSplitQuery()` es una alternativa para ese *shape*, pero debe evaluarse frente al coste de ejecutar varias consultas.

Include sobre referencias

El método `Include` también acepta una expresión lambda que apunta a una propiedad de navegación de referencia. La referencia se carga junto con la entidad principal.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Detalle)
    .Include(o => o.Certificado)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la referencia al detalle. La tercera línea incluye la referencia al certificado. La cuarta línea materializa la consulta.

El SQL generado incluye dos `LEFT JOIN`, uno por cada referencia. Como las referencias no son colecciones, no se produce producto cartesiano.

Error común. si la propiedad de navegación es null, la entidad relacionada no se carga y la propiedad queda a null. Se debe comprobar si la propiedad es null antes de acceder a ella.

ThenInclude

El método `ThenInclude` carga las entidades relacionadas de las entidades incluidas. Se usa después de `Include` para navegar por más de un nivel de relación.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(oa => oa.Aleacion)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de entidades intermedias. La tercera línea incluye la aleación de cada entidad intermedia. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
LEFT JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
ORDER BY [o].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]
```

SQL

La cláusula `SELECT` incluye todas las columnas de las tres tablas. La cláusula `FROM` indica la tabla principal. Las cláusulas `LEFT JOIN` combinan con las tablas relacionadas. El resultado es una lista de órdenes con sus entidades intermedias y sus aleaciones cargadas.

Error común. si se usa `ThenInclude` sin `Include` previo, el compilador lanza un error. `ThenInclude` solo se puede usar después de `Include`.

ThenInclude desde una colección

El método `ThenInclude` se puede usar después de `Include` sobre una colección para cargar las entidades relacionadas de cada elemento de la colección. El parámetro es una expresión lambda que apunta a la propiedad de navegación del elemento de la colección.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ThenInclude(p => p.Orden)
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la orden de cada plancha. La cuarta línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [p].[Id], [p].[Espesor], ..., [o0].[Id], [o0].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [OrdenesFabricacion] AS [o0] ON [p].[OrdenId] = [o0].[Id]
```

SQL

La cláusula SELECT incluye todas las columnas de las tres tablas. La cláusula FROM indica la tabla principal. Las cláusulas LEFT JOIN combinan con las tablas relacionadas. El resultado es una lista de órdenes con sus planchas y la orden de cada plancha cargada.

Error común. si se carga la orden de cada plancha y también se carga la orden principal, se duplica la información. Se debe revisar si es necesario cargar la orden de cada plancha.

Filtered Include

EF Core 5 y versiones posteriores permiten aplicar un filtro a las colecciones incluidas con Include. El filtro se expresa con un Where dentro del Include.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas.Where(p => p.Activa))
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye solo las planchas activas. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [p].[Id], [p].[Espesor], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId] AND [p].[Activa] = CAST(1 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

SQL

La cláusula LEFT JOIN incluye la condición de filtro [p].[Activa] = CAST(1 AS bit). Solo se cargan las planchas activas. Los órdenes sin planchas activas aparecen con la colección vacía.

Error común. si se aplica un filtro a una colección incluida, la colección solo contiene los elementos que cumplen el filtro. Los elementos que no cumplen el filtro no se cargan, aunque existan en la base de datos.

Filtered Include con OrderBy

El filtro de una colección incluida puede incluir un OrderBy para ordenar los elementos de la colección.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas.Where(p => p.Activa).OrderBy(p => p.Espesor))
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye solo las planchas activas y las ordena por espesor. La tercera línea materializa la consulta. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], ..., [p].[Id], [p].[Espesor], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId] AND [p].[Activa] = CAST(1 AS bit)
ORDER BY [o].[Id], [p].[Espesor]
```

SQL

La cláusula LEFT JOIN incluye la condición de filtro. La cláusula ORDER BY ordena por el espesor de las planchas. Solo se cargan las planchas activas, ordenadas por espesor.

Error común. si se aplica un OrderBy a una colección incluida, el orden se aplica a la colección cargada. El orden de las entidades principales no cambia.

AsSplitQuery

`AsSplitQuery()` divide la carga de colecciones incluidas en varias consultas SQL coordinadas. Puede reducir la multiplicación de filas de una consulta única, especialmente con colecciones hermanas, a cambio de más consultas y posibles *roundtrips*.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea divide la consulta en varias. La quinta línea materializa la consulta.

El SQL generado incluye tres consultas: una para las órdenes, una para las planchas y una para las entidades intermedias. Cada consulta se ejecuta por separado y EF Core combina los resultados en memoria.

```
-- Consulta 1: órdenes
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o];

-- Consulta 2: planchas
SELECT [p].[Id], [p].[Espesor], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId];

-- Consulta 3: entidades intermedias
SELECT [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId];
```

SQL

La primera consulta carga las órdenes y las siguientes cargan las colecciones incluidas. EF Core correlaciona los resultados. `AsSplitQuery()` evita la multiplicación de filas entre colecciones hermanas, pero ejecuta varias consultas; la elección debe considerar el volumen de datos, la latencia y los requisitos de consistencia.

Error común. convertir `AsSplitQuery()` en una regla automática. Puede ser útil incluso con una colección grande, y también puede ser peor por los *roundtrips* adicionales. Se debe decidir según la forma y el volumen de la consulta.

AsSingleQuery

`AsSingleQuery()` fuerza la estrategia de consulta única para la carga relacionada. En EF Core es el modo predeterminado cuando no se ha configurado `SplitQuery` globalmente ni se ha aplicado `AsSplitQuery()` a la consulta.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSingleQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea fuerza una sola consulta. La quinta línea materializa la consulta.

En este ejemplo las dos colecciones son hermanas, por lo que el único `SELECT` contiene `JOIN` al mismo nivel y puede multiplicar sus filas relacionadas. `AsSingleQuery()` fuerza esa estrategia de consulta única y debe evaluarse frente a la alternativa dividida.

Error común. asumir que varias colecciones implican siempre el mismo coste. El riesgo de explosión cartesiana aparece especialmente con colecciones hermanas al mismo nivel; hay que comparar `AsSingleQuery()` y `AsSplitQuery()` para el *shape* real.

Configuración global de Split Query

El comportamiento por defecto de las consultas con varios `Include` se puede configurar globalmente en el `DbContext` con `UseSqlServer` y la opción `UseQuerySplittingBehavior`.

CSHARP

```
optionsBuilder.UseSqlServer(
    connectionString,
    sqlOptions => sqlOptions.UseQuerySplittingBehavior(
        QuerySplittingBehavior.SplitQuery));
```

La primera línea configura SQL Server y la segunda establece `SplitQuery` como comportamiento predeterminado para la carga de colecciones relacionadas. Las navegaciones de referencia uno-a-uno siguen resolviéndose mediante `JOIN`, por lo que no conviene resumir esta opción como “todo Include produce otra consulta”.

Error común. establecer `SplitQuery` globalmente sin revisar las consultas concretas. La opción afecta a la estrategia de carga de colecciones y puede añadir *roundtrips*; una consulta puede volver a `AsSingleQuery()` cuando su forma lo justifique.

AutoInclude

`AutoInclude()` configura una navegación en el modelo para que EF Core la incluya automáticamente cuando las consultas devuelven entidades de ese tipo. Se configura en el modelo, por ejemplo desde `OnModelCreating` o una configuración de entidad.

CSHARP

```
modelBuilder.Entity<OrdenFabricacion>()
    .Navigation(o => o.Planchas)
    .AutoInclude();
```

La primera línea selecciona la entidad `OrdenFabricacion`. La segunda selecciona la navegación `Planchas`. La tercera configura la carga automática. Cuando una consulta devuelve entidades `OrdenFabricacion`, EF Core aplica esa navegación automáticamente salvo que se suprima de forma explícita.

La forma SQL concreta depende de la consulta y del modo `single/split`; no se debe asumir que `AutoInclude` equivale siempre a un único `LEFT JOIN`. Puede ser útil para evitar olvidar una relación necesaria, pero también introduce un coste implícito.

Error común. configurar muchos `AutoInclude()` puede hacer que las consultas que devuelven esas entidades carguen relaciones de forma implícita y transfieran más datos de los necesarios. Se debe usar con moderación y revisar el *shape* resultante.

IgnoreAutoIncludes

El método `IgnoreAutoIncludes()` suprime en una consulta concreta las navegaciones configuradas por el usuario con `AutoInclude()`.

CSHARP

```
var ordenes = context.OrdenesFabricacion
    .IgnoreAutoIncludes()
    .ToList();
```

La primera línea inicia la consulta. La segunda línea desactiva la carga automática. La tercera línea materializa la consulta. La colección de planchas no se carga aunque esté configurada con `AutoInclude`.

Error común. usar `IgnoreAutoIncludes()` y asumir que las relaciones necesarias aparecerán igualmente. Si se suprime una carga automática, la consulta debe proyectar o incluir de forma explícita los datos que realmente necesite.

Eager Loading y proyecciones

La carga Eager se puede sustituir o complementar con proyecciones cuando solo se necesita un *shape* de lectura. `Select` permite transferir únicamente las columnas y relaciones proyectadas; no debe afirmarse que toda proyección elimine por sí sola cualquier multiplicación de filas.

CSHARP

```
var resultado = context.OrdenesFabricacion
    .Select(o => new
    {
```

```

o.NumeroOrden,
o.Cliente,
Planchas = o.Planchas
    .Select(p => new { p.Espesor, p.Peso })
    .ToList()
})
.ToList();

```

La primera línea inicia la consulta. La segunda línea proyecta cada orden. La tercera línea incluye el número de orden. La cuarta línea incluye el cliente. La quinta línea proyecta la colección de planchas. La sexta línea materializa la consulta. El SQL generado incluye un LEFT JOIN con la tabla de planchas. El resultado es una lista de tipos anónimos con las planchas proyectadas.

Error común. si el DTO de destino declara una propiedad `List<T>`, la proyección debe producir una lista compatible; `ToList()` dentro de la proyección hace explícita esa forma. No es una regla universal para toda proyección de colecciones, sino una exigencia del shape elegido.

El proyecto AceriaData

En el proyecto `AceriaData`, se añaden métodos de carga Eager al repositorio de órdenes. Los métodos usan `Include`, `ThenInclude`, `Filtered Include` y `AsSplitQuery`. La capa de aplicación usa estos métodos a través de la unidad de trabajo. La base de datos sigue siendo `AceriaDB` en `SQL Server LocalDB`.

Validación técnica en AceriaData

En 3.8 se configura `AutoInclude()` sobre `OrdenFabricacion.Planchas` para demostrar su efecto real y se contrasta con `IgnoreAutoIncludes()`. La configuración se conserva durante 3.9–3.11 para que pueda observarse y aislarse explícitamente; en 3.12 se retira como decisión final de diseño para evitar un coste de carga oculto en el contrato de acceso a datos.

En el checkpoint `M03/PROYECTO0/3.8`, esta materia se ejecuta sobre `SQL Server LocalDB` después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el `Change Tracker` antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `Include`, `ThenInclude`, `Filtered Include` con `AsNoTracking()`, `AsSplitQuery()` y `AutoInclude()` configurado en el modelo y contrastado con `IgnoreAutoIncludes()`. El punto termina con el marcador `E2E 3.8 OK`.

Resumen de la teoría

- La carga Eager solicita de antemano las relaciones necesarias; puede ejecutarse como consulta única o dividirse con `AsSplitQuery()`.
- `Include` carga colecciones y referencias.
- `ThenInclude` carga relaciones anidadas.
- `Filtered Include` permite filtrar las colecciones incluidas.
- `AsSplitQuery()` divide la carga de colecciones en varias consultas y puede evitar la multiplicación de filas de colecciones hermanas.
- `AsSingleQuery` fuerza una sola consulta.
- `AutoInclude` configura la carga automática de una propiedad de navegación.
- `IgnoreAutoIncludes()` suprime los `AutoInclude` configurados por el usuario para esa consulta.
- La carga Eager se combina con proyecciones.
- En el proyecto `AceriaData` se añaden métodos de carga Eager al repositorio.

Punto 3.9 — Lazy Loading: configuración, funcionamiento y riesgos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se explora la carga Lazy (carga diferida) en el proyecto AceriaData, incluyendo su configuración con proxies, su funcionamiento interno, el problema N+1 y sus riesgos en aplicaciones reales.

Objetivos de aprendizaje

- Comprender qué es la carga Lazy y cómo se diferencia de la carga Eager.
- Configurar la carga Lazy con proxies en EF Core.
- Comprender cómo funciona el proxy y cuándo se activa la carga.
- Identificar el problema N+1 que provoca la carga Lazy.
- Comparar el rendimiento de la carga Lazy con la carga Eager.
- Comprender los riesgos de la carga Lazy en aplicaciones web y servicios.
- Aplicar la carga Lazy al proyecto AceriaData con fines demostrativos.

Teoría

Qué es la carga Lazy

La carga Lazy (carga diferida) es la técnica que carga las entidades relacionadas en el momento en que se accede a la propiedad de navegación, no antes. La entidad relacionada no se carga cuando se carga la entidad principal, sino cuando el código accede a la propiedad de navegación por primera vez. En ese momento, EF Core ejecuta una consulta adicional contra la base de datos para cargar la entidad relacionada.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
Console.WriteLine(orden!.Cliente);
Console.WriteLine(orden.Planchas.Count);
```

CSHARP

La primera línea carga la orden sin sus planchas. La segunda línea accede a la propiedad Cliente, que ya está cargada. La tercera línea accede a la propiedad Planchas. Si la carga Lazy está habilitada, EF Core ejecuta una consulta adicional para cargar las planchas de esa orden en ese momento.

La carga Lazy es cómoda porque no hay que especificar qué entidades relacionadas se quieren cargar: se cargan bajo demanda. Sin embargo, esta comodidad tiene un precio: se ejecutan más consultas de las necesarias, lo que puede degradar el rendimiento.

Diferencia entre carga Eager y carga Lazy

La carga Eager solicita de antemano las relaciones previstas y puede resolverse en una consulta única o con `AsSplitQuery()`. Lazy Loading obtiene una navegación cuando se accede a ella si aún no está cargada y el contexto permite hacerlo. Cuando se sabe qué datos relacionados se necesitan, Eager Loading o una proyección suelen ofrecer un patrón más predecible; Lazy Loading prioriza comodidad a cambio de *roundtrips* potencialmente ocultos.

```
// Carga Eager: una sola consulta con LEFT JOIN
var ordenes1 = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

// Carga Lazy: una consulta para las órdenes y otra por cada orden al acceder a Planchas
var ordenes2 = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes2)
{
    Console.WriteLine(orden.Planchas.Count);
}
```

CSHARP

En este ejemplo, el primer patrón usa Eager Loading de una sola colección en modo single-query. El segundo, suponiendo proxies Lazy habilitados, navegación inicialmente no cargada y sin `AutoInclude`, carga primero las órdenes y puede lanzar una consulta adicional por cada orden al acceder por primera vez a `Planchas`; con cien órdenes podría llegar a 101 consultas.

Error común. si se usa carga Lazy en un bucle sin entender el problema N+1, se ejecutan muchas consultas contra la base de datos. Se debe preferir la carga Eager cuando se sabe que se van a necesitar las entidades relacionadas.

Configurar la carga Lazy con proxies

EF Core soporta la carga Lazy a través de proxies. Los proxies son clases que heredan de las entidades y sobrescriben las propiedades de navegación para interceptar el acceso y cargar la entidad relacionada. Para usar proxies, se necesita el paquete `Microsoft.EntityFrameworkCore.Proxies` y configurar la opción `UseLazyLoadingProxies`.

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies
```

BASH

```
optionsBuilder
    .UseSqlServer(connectionString)
    .UseLazyLoadingProxies();
```

CSHARP

La primera línea añade el paquete de proxies. La segunda línea configura el proveedor de SQL Server. La tercera línea habilita la carga Lazy con proxies. A partir de este momento, todas las propiedades de navegación se cargan bajo demanda si están marcadas como virtual.

Requisitos para los proxies

Para que los proxies funcionen, las propiedades de navegación deben ser virtual. Las propiedades virtual permiten que el proxy las sobrescriba y añada la lógica de carga Lazy. Además, las clases de entidad no pueden ser sealed y las propiedades de navegación deben ser accesibles.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;

    public virtual List<PlanchaAcero> Planchas { get; set; } = new();
    public virtual DetalleOrden? Detalle { get; set; }
}
```

CSHARP

La primera línea declara la clase. Las siguientes líneas declaran las propiedades escalares. La penúltima línea declara la colección de planchas como virtual. La última línea declara la referencia al detalle como virtual. El proxy sobrescribe estas propiedades para interceptar el acceso.

Error común. con proxies, las navegaciones que se quieran cargar de forma Lazy deben poder ser sobrescritas por el proxy —habitualmente se declaran `virtual`—. No significa que todas las navegaciones del modelo deban hacerse `virtual` si no se pretende usar Lazy Loading en ellas.

Cómo funciona el proxy

Cuando EF Core carga una entidad con la carga Lazy habilitada, no crea una instancia de la clase original, sino una instancia de una clase derivada generada dinámicamente: el proxy. El proxy hereda de la clase original y sobrescribe las propiedades de navegación. Cuando el código accede a una propiedad de navegación, el proxy comprueba si la entidad relacionada ya está cargada. Si no lo está, ejecuta una consulta contra la base de datos para cargarla y la almacena en la propiedad.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
Console.WriteLine(orden!.GetType().Name);
```

CSHARP

La primera línea carga la orden. La segunda línea imprime el nombre del tipo. El tipo es algo como `OrdenFabricacionProxy`, que es la clase generada por EF Core.

Error común. si se usan serializadores que no soportan proxies, se pueden producir errores al serializar las entidades. Se debe configurar el serializador para que ignore las propiedades del proxy.

El problema N+1

El problema N+1 es el problema de rendimiento más común asociado a la carga Lazy. Consiste en ejecutar una consulta para cargar las entidades principales y después una consulta por cada entidad principal para cargar sus entidades relacionadas. Si hay N entidades principales, se ejecutan N+1 consultas.

CSHARP

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    Console.WriteLine($"Orden {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
}
```

La primera línea carga todas las órdenes. El bucle itera sobre las órdenes. En cada iteración, se accede a `orden.Planchas`, lo que provoca una consulta adicional. Si hay cien órdenes, se ejecutan ciento una consultas: una para las órdenes y cien para las planchas.

Error común. con Lazy Loading habilitado, si la navegación aún no está cargada, acceder a ella dentro de un bucle puede disparar una consulta adicional por entidad y producir N+1. Si se sabe que la relación será necesaria, conviene planificar la carga Eager o una proyección adecuada.

El problema N+1 en la práctica

El problema N+1 es especialmente problemático en aplicaciones web y servicios, donde el tiempo de respuesta es crítico. Cada consulta adicional añade latencia y consume recursos de la base de datos. En una aplicación con muchas peticiones concurrentes, el problema N+1 puede provocar la saturación de la base de datos.

CSHARP

```
// Anti-patrón: N+1
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var planchas = orden.Planchas;
}

// Buen patrón: una sola consulta
var ordenesConPlanchas = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();
```

Con Lazy Loading habilitado y las navegaciones inicialmente sin cargar, el primer patrón puede ejecutar N+1 consultas. El segundo expresa Eager Loading; con una sola colección y el modo single-query usado en este ejemplo se resuelve con una consulta. La segunda es mucho más eficiente.

Error común. si se usa carga Lazy en un bucle, el problema N+1 puede pasar desapercibido hasta que la aplicación se pone en producción con datos reales. Se debe revisar el código y usar Include cuando sea posible.

La carga Lazy en aplicaciones web

La carga Lazy es especialmente peligrosa en aplicaciones web porque el `DbContext` se registra con ciclo de vida Scoped y se libera al final de la petición. Si se accede a una propiedad de navegación después de que el `DbContext` se haya liberado, se produce una excepción `ObjectDisposedException`.

CSHARP

```
public IActionResult ObtenerOrden(int id)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);
    return Ok(orden);
}

public IActionResult ObtenerPlanchas(int id)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);
    return Ok(orden.Planchas);
}
```

El primer método devuelve la orden sin sus planchas. El segundo método devuelve las planchas. Si el primer método se ejecuta y después se accede a `orden.Planchas` fuera del ámbito del `DbContext`, se produce una excepción.

Error común. serializar directamente entidades con Lazy Loading puede acceder a navegaciones de forma implícita. Si el `DbContext` sigue disponible, eso puede disparar consultas inesperadas; si ya fue dispuesto, la carga no puede completarse. En APIs suele ser preferible proyectar a DTOs y controlar explícitamente qué datos se serializan.

La carga Lazy en la serialización

La carga Lazy es especialmente problemática en la serialización JSON. Cuando se serializa una entidad, el serializador accede a todas sus propiedades, incluyendo las propiedades de navegación. Si la carga Lazy está habilitada, cada propiedad de navegación provoca una consulta adicional. Esto puede provocar el problema N+1 durante la serialización.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var json = JsonSerializer.Serialize(orden);
```

CSHARP

La primera línea carga la orden. La segunda línea serializa la orden. Durante la serialización, se accede a las propiedades de navegación, lo que provoca consultas adicionales. Además, si las entidades relacionadas tienen propiedades de navegación hacia la orden, se produce una referencia circular que provoca un desbordamiento de pila.

Error común. si se serializan entidades con propiedades de navegación y carga Lazy, se pueden producir errores de serialización. Se debe usar DTOs para la serialización o deshabilitar la carga Lazy.

La referencia circular

La referencia circular es un problema que ocurre cuando dos entidades se referencian mutuamente. Si se serializan ambas, el serializador entra en un bucle infinito. La carga Lazy agrava este problema porque carga ambas entidades al acceder a sus propiedades.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public virtual List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera clase tiene una colección de planchas. La segunda clase tiene una referencia a la orden. Si se serializa una orden, el serializador accede a las planchas. Si se serializa una plancha, el serializador accede a la orden. El ciclo se repite indefinidamente.

Error común. si se serializan entidades con referencias circulares, se produce un `JsonException` por referencia circular. Se debe usar DTOs o configurar el serializador para ignorar las referencias circulares.

La carga Lazy en aplicaciones de consola

En el ejemplo de consola de `AceriaData`, el `DbContext` permanece vivo durante la operación, por lo que el proxy puede cargar la navegación al acceder a ella. Esto no es una propiedad automática de todas las aplicaciones de consola: el comportamiento depende del ciclo de vida que se dé al contexto. El riesgo N+1 sigue existiendo.

```
using var context = new AceriaDbContext();
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    Console.WriteLine(orden.Planchas.Count);
}
```

CSHARP

En este ejemplo, con proxies habilitados y las navegaciones inicialmente sin cargar, la primera consulta obtiene los órdenes y cada primer acceso a `Planchas` puede lanzar otra consulta. Como el `DbContext` sigue vivo, el patrón puede convertirse en N+1.

Error común. recorrer muchas entidades y activar navegaciones Lazy puede multiplicar los *roundtrips*. Si la relación se conoce de antemano, conviene planificar Eager Loading o una proyección; esa carga puede ser single-query o split-query según la forma elegida.

Deshabilitar la carga Lazy

Con proxies, la forma más clara de no usar Lazy Loading es no configurar `UseLazyLoadingProxies()`; además, una navegación que el proxy no puede sobrescribir no se cargará mediante ese mecanismo. `IgnoreAutoIncludes()` no deshabilita Lazy Loading: solo suprime los `AutoInclude()` configurados en el modelo. En *AceriaData* se usa precisamente para que el `AutoInclude` heredado de 3.8 no oculte la demostración Lazy de 3.9.

```
// Sin carga Lazy
```

CSHARP

```
optionsBuilder
```

```
.UseSqlServer(connectionString);
```

La primera línea configura el proveedor de SQL Server sin habilitar la carga Lazy. A partir de este momento, las propiedades de navegación no se cargan bajo demanda. Se debe usar `Include` para cargarlas.

Error común. deshabilitar Lazy Loading y esperar que una navegación se cargue por sí sola. Sin Eager Loading, Explicit Loading o una proyección que la obtenga, EF Core no ejecutará automáticamente una consulta para esa relación; el valor observado dependerá de cómo esté inicializada la navegación y de lo que ya esté rastreado.

Cuándo usar la carga Lazy

La carga Lazy puede ser útil en escenarios controlados donde el ciclo de vida del `DbContext` es conocido y los accesos adicionales son aceptables. No es el tipo de aplicación —consola, escritorio o web— lo que la hace apropiada por sí solo: hay que valorar el número de accesos, los *roundtrips* y la previsibilidad de la consulta.

La carga Lazy no es adecuada en aplicaciones web, servicios de alta concurrencia o cualquier escenario donde el rendimiento sea crítico. En estos casos, se debe usar la carga Eager con `Include` o la carga Explicit con `Entry().Collection().Load()`.

El proyecto AceriaData

En el proyecto *AceriaData*, se explora la carga Lazy con fines demostrativos. Se habilita la carga Lazy con proxies, se marcan las propiedades de navegación como virtual y se demuestra el problema N+1. Se comparan los resultados con la carga Eager y se analiza el SQL generado. La base de datos sigue siendo *AceriaDB* en SQL Server LocalDB.

Nota. la carga Lazy se usa solo con fines demostrativos en este punto. En los módulos posteriores se usará la carga Eager para evitar el problema N+1.

Validación técnica en AceriaData

La carga Lazy se habilita **solo en 3.9**. El acceso a `orden.Planchas` dentro de un bucle demuestra el patrón que puede originar N+1; el curso no presenta Lazy Loading como estrategia por defecto. En 3.10 se retira `UseLazyLoadingProxies()`.

En el checkpoint `M03/PROYECTO/3.9`, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `Microsoft.EntityFrameworkCore.Proxies`, `UseLazyLoadingProxies()`, navegaciones virtual, riesgo N+1 demostrado al acceder en bucle, proxies limitados al checkpoint 3.9. El punto termina con el marcador `E2E 3.9 OK`.

Resumen de la teoría

- La carga Lazy carga las entidades relacionadas cuando se accede a ellas.
- Se configura con `UseLazyLoadingProxies`.
- Requiere el paquete `Microsoft.EntityFrameworkCore.Proxies`.
- Las propiedades de navegación deben ser virtual.
- El proxy sobrescribe las propiedades de navegación para interceptar el acceso.
- El problema N+1 consiste en ejecutar N+1 consultas por acceder a las propiedades de navegación en un bucle.
- La carga Lazy es peligrosa en aplicaciones web por el ciclo de vida del `DbContext`.

- La carga Lazy es problemática en la serialización por las referencias circulares.
- La carga Lazy se puede deshabilitar no configurando los proxies.
- En el proyecto AceriaData se explora la carga Lazy con fines demostrativos.

Punto 3.10 — Explicit Loading

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se explora la carga Explicit (carga explícita) en el proyecto AceriaData, incluyendo la carga de colecciones y referencias con `Entry().Collection().Load()` y `Entry().Reference().Load()`, así como la carga condicional con `Query()`.

Objetivos de aprendizaje

- Comprender qué es la carga Explicit y cuándo usarla.
- Cargar referencias con `Entry().Reference().Load()`.
- Cargar colecciones con `Entry().Collection().Load()`.
- Aplicar filtros a las colecciones cargadas con `Query()`.
- Comprobar si una propiedad de navegación ya está cargada con `IsLoaded`.
- Combinar la carga Explicit con la carga Eager.
- Analizar el SQL generado por las consultas de carga Explicit.
- Aplicar la carga Explicit al proyecto AceriaData.

Teoría

Qué es la carga Explicit

La carga Explicit (carga explícita) es la técnica que carga las entidades relacionadas de forma manual y controlada. A diferencia de la carga Eager, que carga las entidades relacionadas en la consulta principal, y de la carga Lazy, que las carga automáticamente al acceder a ellas, la carga Explicit las carga cuando el código lo indica explícitamente.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden!).Collection(o => o.Planchas).Load();
```

CSHARP

La primera línea carga la orden sin sus planchas. La segunda línea carga la colección de planchas de forma explícita. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Id] = 1;

SELECT [p].[Id], [p].[Espesor], [p].[Peso], ...
FROM [PlanchasAcero] AS [p]
WHERE [p].[OrdenId] = 1;
```

SQL

La primera consulta carga la orden. La segunda consulta carga las planchas de esa orden. La carga Explicit ejecuta una consulta adicional por cada propiedad de navegación que se carga.

La carga Explicit es útil cuando no se sabe de antemano qué entidades relacionadas se van a necesitar y no se quiere usar la carga Lazy. Permite cargar las entidades relacionadas bajo demanda de forma controlada, evitando el problema N+1 porque el programador decide cuándo cargar.

Cargar referencias con Reference

El método `Entry().Reference().Load()` carga una propiedad de navegación de referencia. Se usa cuando la entidad relacionada es una sola entidad, no una colección.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden!).Reference(o => o.Detalle).Load();
```

CSHARP

La primera línea carga la orden. La segunda línea carga la referencia al detalle. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Id] = 1;

SELECT [d].[Id], [d].[ComposicionQuimica], [d].[TemperaturaColada], ...
FROM [DetallesOrden] AS [d]
WHERE [d].[OrdenId] = 1;
```

SQL

La primera consulta carga la orden. La segunda consulta carga el detalle de esa orden. Si la orden no tiene detalle, la propiedad Detalle queda a null.

Error común. si se llama a Load sobre una referencia que ya está cargada, EF Core no ejecuta ninguna consulta adicional. Se puede comprobar si está cargada con IsLoaded.

Cargar colecciones con Collection

El método Entry().Collection().Load() carga una propiedad de navegación de colección. Se usa cuando la entidad relacionada es una colección de entidades.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden!).Collection(o => o.Planchas).Load();
```

CSHARP

La primera línea carga la orden. La segunda línea carga la colección de planchas. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Id] = 1;

SELECT [p].[Id], [p].[Espesor], [p].[Peso], ...
FROM [PlanchasAcero] AS [p]
WHERE [p].[OrdenId] = 1;
```

SQL

La primera consulta carga la orden. La segunda consulta carga las planchas de esa orden. Si la orden no tiene planchas, la colección queda vacía.

Error común. si se llama a Load sobre una colección que ya está cargada, EF Core no ejecuta ninguna consulta adicional. Se puede comprobar si está cargada con IsLoaded.

Comprobar si una propiedad está cargada

El método IsLoaded comprueba si una propiedad de navegación ya está cargada. Se usa para evitar cargas innecesarias.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var entry = context.Entry(orden!);

if (!entry.Collection(o => o.Planchas).IsLoaded)
{
    entry.Collection(o => o.Planchas).Load();
}
```

CSHARP

La primera línea carga la orden. La segunda línea obtiene el EntityEntry. La tercera línea comprueba si la colección ya está cargada. La cuarta línea carga la colección solo si no está cargada.

Error común. asumir que Load() vuelve a consultar una navegación que EF Core ya conoce como cargada. Cuando IsLoaded es true, las llamadas posteriores a Load()/LoadAsync() son un no-op. Consultar IsLoaded sigue siendo útil para hacer explícita la intención y para entender casos donde EF no puede asegurar que la colección esté completamente cargada.

Cargar colecciones con filtro

El método Query() permite aplicar un filtro a la colección antes de cargarla. Se combina con Load para cargar solo los elementos que cumplen el filtro.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden!).Collection(o => o.Planchas).Query().Where(p => p.Activa).Load();
```

CSHARP

La primera línea carga la orden. La segunda línea carga solo las planchas activas de esa orden. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Id] = 1;

SELECT [p].[Id], [p].[Espesor], [p].[Peso], ...
FROM [PlanchasAcero] AS [p]
WHERE [p].[OrdenId] = 1 AND [p].[Activa] = CAST(1 AS bit);
```

SQL

La primera consulta carga la orden. La segunda consulta carga solo las planchas activas. Las planchas inactivas no se cargan.

Error común. si se aplica un filtro a una colección cargada con Query, la colección solo contiene los elementos que cumplen el filtro. Los elementos que no cumplen el filtro no se cargan, aunque existan en la base de datos.

Cargar colecciones con ordenación

El método Query() también permite ordenar la colección antes de cargarla. Se combina con OrderBy y Load.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden!).Collection(o => o.Planchas).Query().OrderBy(p => p.Espesor).Load();
```

CSHARP

La primera línea carga la orden. La segunda línea carga las planchas ordenadas por espesor. El SQL generado tiene la siguiente forma:

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Id] = 1;

SELECT [p].[Id], [p].[Espesor], [p].[Peso], ...
FROM [PlanchasAcero] AS [p]
WHERE [p].[OrdenId] = 1
ORDER BY [p].[Espesor];
```

SQL

La primera consulta carga la orden. La segunda consulta carga las planchas ordenadas por espesor.

Error común. si se aplica un OrderBy a una colección cargada con Query, el orden se aplica a la colección cargada. El orden de las entidades principales no cambia.

Cargar referencias con filtro

El método Query() también se aplica a las referencias. Sin embargo, como una referencia es una sola entidad, el filtro se aplica a esa entidad.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden!).Reference(o => o.Detalle).Query().Where(d => d.TemperaturaColada > 1500).Load();
```

CSHARP

La primera línea carga la orden. La segunda línea carga el detalle solo si la temperatura de colada es mayor que 1500. Si el detalle no cumple el filtro, la propiedad Detalle queda a null.

Error común. si se aplica un filtro a una referencia y la entidad no cumple el filtro, la propiedad queda a null. Se debe comprobar antes de acceder a ella.

Cargar varias propiedades de navegación

Se pueden cargar varias propiedades de navegación de forma explícita. Cada llamada a Load ejecuta una consulta adicional.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var entry = context.Entry(orden!);

entry.Collection(o => o.Planchas).Load();
entry.Reference(o => o.Detalle).Load();
entry.Collection(o => o.OrdenesAleaciones).Load();
```

CSHARP

La primera línea carga la orden. La segunda línea obtiene el EntityEntry. La tercera línea carga las planchas. La cuarta línea carga el detalle. La quinta línea carga las entidades intermedias. Se ejecutan tres consultas adicionales.

Error común. si se cargan varias propiedades de navegación con Load, se ejecutan varias consultas adicionales. Se debe usar Include si se van a cargar varias propiedades en el mismo contexto.

Cargar propiedades de navegación de entidades relacionadas

Se puede cargar una propiedad de navegación de una entidad relacionada. Se accede al EntityEntry de la entidad relacionada y se llama a Load.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden!).Collection(o => o.Planchas).Load();

foreach (var plancha in orden!.Planchas)
{
    context.Entry(plancha).Reference(p => p.Orden).Load();
}
```

CSHARP

La primera línea carga la orden. La segunda línea carga las planchas. El bucle itera sobre las planchas. La cuarta línea carga la orden de cada plancha. Como la orden ya está cargada, EF Core no ejecuta consultas adicionales.

Error común. si se carga la orden de cada plancha y la orden ya está cargada, EF Core devuelve la instancia existente. Si la orden no estuviera cargada, se ejecutaría una consulta por cada plancha.

Comparación entre carga Eager, Lazy y Explicit

Las tres técnicas tienen ventajas y desventajas. Eager Loading solicita de antemano las relaciones y puede ejecutarse como consulta única o dividida. Lazy Loading puede cargar relaciones automáticamente al acceder a ellas y provocar N+1. Explicit Loading permite decidir posteriormente qué navegación cargar, pero normalmente implica operaciones adicionales y más código.

```
// Carga Eager
var orden1 = context.OrdenesFabricacion.Include(o => o.Planchas).FirstOrDefault(o => o.Id == 1);

// Carga Lazy
var orden2 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var planchas2 = orden2!.Planchas;

// Carga Explicit
var orden3 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.Entry(orden3!).Collection(o => o.Planchas).Load();
```

CSHARP

La primera consulta usa carga Eager. La segunda usa carga Lazy. La tercera usa carga Explicit. Las tres devuelven el mismo resultado, pero con diferente número de consultas y diferente código.

Error común. si se mezclan las tres técnicas sin criterio, el código se vuelve confuso y difícil de mantener. Se debe elegir una técnica y usarla de forma consistente.

Cuándo usar la carga Explicit

La carga Explicit es adecuada cuando no se sabe de antemano qué entidades relacionadas se van a necesitar y no se quiere usar la carga Lazy. También es adecuada cuando se quiere cargar una propiedad de navegación solo si se cumple una condición. Y cuando se quiere cargar una propiedad de navegación después de haber cargado la entidad principal.

Si se sabe de antemano que varias relaciones serán necesarias, Eager Loading o una proyección suelen ser más simples y pueden reducir *roundtrips* frente a varias cargas explícitas. Explicit Loading sigue siendo válida cuando interesa decidir posteriormente qué navegación cargar o aplicar un filtro específico; la elección debe medirse para el escenario real.

El proyecto AceriaData

En el proyecto AceriaData, se añaden métodos de carga Explicit al repositorio de órdenes. Los métodos usan `Entry().Collection().Load()` y `Entry().Reference().Load()` para cargar las entidades relacionadas bajo demanda. La capa de aplicación usa estos métodos a través de la unidad de trabajo. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Nota. la carga Explicit se usa en este punto con fines demostrativos. En los módulos posteriores se usará la carga Eager para evitar el problema N+1.

Validación técnica en AceriaData

En el checkpoint `M03/PROYECTO/3.10`, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `Entry().Collection().Load()`, `Entry().Reference().Load()`, `IsLoaded`, `Query()` antes de `Load()`, Lazy Loading desactivado. El punto termina con el marcador `E2E 3.10 OK`.

Resumen de la teoría

- La carga Explicit carga las entidades relacionadas de forma manual y controlada.
- Se usa `Entry().Reference().Load()` para cargar referencias.
- Se usa `Entry().Collection().Load()` para cargar colecciones.
- Se usa `IsLoaded` para comprobar si una propiedad ya está cargada.
- Se usa `Query()` para aplicar filtros y ordenaciones antes de cargar.
- Cada `Load()` explícito de una navegación que aún no esté cargada puede ejecutar una consulta adicional.
- La carga Explicit se combina con la carga Eager.
- La carga Explicit es adecuada cuando no se sabe de antemano qué se va a necesitar.
- En el proyecto AceriaData se añaden métodos de carga Explicit al repositorio.

Punto 3.11 — Composición de consultas y ejecución diferida

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en la composición de consultas y la ejecución diferida en el proyecto AceriaData, incluyendo la construcción incremental de consultas, las consultas condicionales y la reutilización de consultas base.

Objetivos de aprendizaje

- Comprender qué es la composición de consultas y por qué es útil.
- Construir consultas de forma incremental con IQueryable.
- Aplicar filtros y ordenaciones de forma condicional.
- Reutilizar consultas base con diferentes filtros.
- Comprender la ejecución diferida y cuándo se materializa una consulta.
- Evitar la materialización prematura y el problema N+1.
- Analizar el SQL generado por consultas compuestas.
- Aplicar la composición de consultas al proyecto AceriaData.

Teoría

Qué es la composición de consultas

La composición de consultas es la técnica que consiste en construir una consulta de forma incremental, añadiendo filtros, ordenaciones y proyecciones paso a paso. Cada operador LINQ devuelve un nuevo IQueryable que se puede seguir componiendo. La consulta no se ejecuta hasta que se materializa.

```
var consulta = context.OrdenesFabricacion.AsQueryable();

if (!string.IsNullOrEmpty(cliente))
{
    consulta = consulta.Where(o => o.Cliente == cliente);
}

if (!string.IsNullOrEmpty(estado))
{
    consulta = consulta.Where(o => o.Estado == estado);
}

var resultados = consulta.ToList();
```

CSHARP

La primera línea inicia la consulta. Las siguientes líneas añaden filtros condicionales. La última línea materializa la consulta. Solo se ejecuta un SELECT con todos los filtros aplicados. La composición de consultas es útil cuando los filtros dependen de parámetros opcionales.

La ejecución diferida

La ejecución diferida es el comportamiento por el que una consulta LINQ no se ejecuta hasta que se itera sobre ella o se materializa. Esto permite componer consultas de forma incremental sin ejecutar nada hasta el final.

```
var consulta = context.OrdenesFabricacion.Where(o => o.Cliente == "Constructora del Norte");

// La consulta no se ha ejecutado todavía

var lista = consulta.ToList();

// La consulta se ha ejecutado con el filtro por cliente
```

CSHARP

La primera línea construye la consulta. La segunda línea es un comentario. La tercera línea materializa la consulta. Solo en la tercera línea se ejecuta el SQL.

La ejecución diferida permite reutilizar una consulta base con diferentes filtros. Cada vez que se materializa la consulta, se ejecuta con los filtros que se hayan añadido hasta ese momento.

```
var consultaBase = context.OrdenesFabricacion.AsQueryable();

var pendientes = consultaBase.Where(o => o.Estado == "Pendiente").ToList();
var enProceso = consultaBase.Where(o => o.Estado == "EnProceso").ToList();
var todas = consultaBase.ToList();
```

CSHARP

La primera línea inicia la consulta base. La segunda línea materializa las órdenes pendientes. La tercera línea materializa las órdenes en proceso. La cuarta línea materializa todas las órdenes. Cada materialización ejecuta una consulta distinta con los filtros correspondientes.

Error común. si se modifica la consulta base después de materializarla, la modificación no afecta a la lista ya materializada. La consulta base sigue siendo la misma.

Composición con Where condicional

El operador Where se puede aplicar de forma condicional. Si la condición no se cumple, el filtro no se aplica.

```
var consulta = context.OrdenesFabricacion.AsQueryable();

if (fechaDesde.HasValue)
{
    consulta = consulta.Where(o => o.FechaCreacion >= fechaDesde.Value);
}

if (fechaHasta.HasValue)
{
    consulta = consulta.Where(o => o.FechaCreacion <= fechaHasta.Value);
}

var resultados = consulta.ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea comprueba si la fecha desde tiene valor. La tercera línea añade el filtro por fecha desde. La cuarta línea comprueba si la fecha hasta tiene valor. La quinta línea añade el filtro por fecha hasta. La sexta línea materializa la consulta. El SQL generado incluye solo los filtros que se han aplicado.

Error común. insertar ToList() en mitad de la composición cambia los operadores posteriores a LINQ to Objects y obliga a transferir antes los datos. Solo habrá varias consultas de base de datos si además se vuelven a ejecutar otros operadores terminales sobre un IQueryable; el problema principal de la materialización temprana es perder composición en servidor.

Composición con OrderBy condicional

El operador OrderBy se puede aplicar de forma condicional. Si no se aplica, el orden es el que devuelve la base de datos.

```
var consulta = context.OrdenesFabricacion.AsQueryable();

if (ordenarPorCliente)
{
    consulta = consulta.OrderBy(o => o.Cliente);
}
else
{
    consulta = consulta.OrderBy(o => o.FechaCreacion);
}

var resultados = consulta.ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea comprueba si se debe ordenar por cliente. La tercera línea ordena por cliente. La cuarta línea indica el caso contrario. La quinta línea ordena por fecha. La sexta línea materializa la consulta. El SQL generado incluye el ORDER BY correspondiente.

Error común. si se aplica OrderBy dos veces, el segundo sobrescribe al primero. Para añadir un criterio secundario, se debe usar ThenBy.

Composición con ThenBy condicional

El operador ThenBy se puede aplicar de forma condicional para añadir un criterio de ordenación secundario.

```
var consulta = context.OrdenesFabricacion
    .OrderBy(o => o.Cliente)
    .AsQueryable();

if (ordenarPorFecha)
{
    consulta = consulta.ThenBy(o => o.FechaCreacion);
}
```

CSHARP

```

    consulta = ((IOrderedQueryable<OrdenFabricacion>)consulta).ThenBy(o => o.FechaCreacion);
}

var resultados = consulta.ToList();

```

La primera línea inicia la consulta. La segunda línea ordena por cliente. La tercera línea convierte la consulta a IQueryable. La cuarta línea comprueba si se debe ordenar por fecha. La quinta línea añade el criterio secundario. La sexta línea materializa la consulta.

Error común. si se convierte la consulta a IQueryable después de OrderBy, se pierde la capacidad de usar ThenBy. Se debe mantener el tipo IOrderedQueryable para poder usar ThenBy.

Composición con Skip y Take

Los operadores Skip y Take se pueden aplicar de forma condicional para paginar los resultados.

```

var consulta = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .AsQueryable();

if (pagina > 0 && tamanoPagina > 0)
{
    consulta = consulta.Skip((pagina - 1) * tamanoPagina).Take(tamanoPagina);
}

var resultados = consulta.ToList();

```

CSHARP

La primera línea inicia la consulta. La segunda línea ordena por fecha. La tercera línea convierte la consulta a IQueryable. La cuarta línea comprueba si se debe paginar. La quinta línea aplica Skip y Take. La sexta línea materializa la consulta. El SQL generado incluye OFFSET y FETCH en SQL Server.

Error común. si se aplica Skip sin OrderBy, el resultado es indeterminado. Se debe aplicar OrderBy antes de Skip.

Composición con Select condicional

El operador Select se puede aplicar de forma condicional para proyectar los resultados a diferentes formas.

```

var consulta = context.OrdenesFabricacion.AsQueryable();

if (soloResumen)
{
    var resmenes = consulta.Select(o => new { o.NumeroOrden, o.Cliente }).ToList();
}
else
{
    var ordenes = consulta.ToList();
}

```

CSHARP

La primera línea inicia la consulta. La segunda línea comprueba si se quiere solo el resumen. La tercera línea proyecta a un tipo anónimo y materializa. La cuarta línea indica el caso contrario. La quinta línea materializa las órdenes completas.

Error común. tratar el orden Where/Select como una regla de rendimiento absoluta. EF Core puede traducir un filtro posterior si la proyección conserva los datos necesarios. Filtrar antes de proyectar suele facilitar la lectura y la composición, pero el criterio definitivo es el SQL generado.

Reutilización de consultas base

Una consulta base se puede reutilizar con diferentes filtros. Cada materialización ejecuta la consulta con los filtros aplicados hasta ese momento.

```

public IQueryable<OrdenFabricacion> ObtenerConsultaBase()
{

```

CSHARP

```

    return context.OrdenesFabricacion.AsQueryable();
}

var consultaBase = ObtenerConsultaBase();
var pendientes = consultaBase.Where(o => o.Estado == "Pendiente").ToList();
var enProceso = consultaBase.Where(o => o.Estado == "EnProceso").ToList();

```

La primera línea declara el método que devuelve la consulta base. La segunda línea devuelve la consulta base. La tercera línea obtiene la consulta base. La cuarta línea materializa las órdenes pendientes. La quinta línea materializa las órdenes en proceso.

Error común. materializar la consulta base antes de terminar la composición introduce trabajo innecesario. Dentro de Infrastructure se puede conservar IQueryable hasta el operador terminal, pero el checkpoint 3.12 demuestra por qué ese IQueryable no debe exponerse como contrato público de Application.

Composición con Include condicional

El operador Include se puede aplicar de forma condicional para cargar entidades relacionadas según sea necesario.

```

var consulta = context.OrdenesFabricacion.AsQueryable();

if (incluirPlanchas)
{
    consulta = consulta.Include(o => o.Planchas);
}

if (incluirDetalle)
{
    consulta = consulta.Include(o => o.Detalle);
}

var resultados = consulta.ToList();

```

CSHARP

La primera línea inicia la consulta. La segunda línea comprueba si se deben incluir las planchas. La tercera línea incluye las planchas. La cuarta línea comprueba si se debe incluir el detalle. La quinta línea incluye el detalle. La sexta línea materializa la consulta. El SQL generado incluye los LEFT JOIN correspondientes.

Error común. varias colecciones hermanas incluidas al mismo nivel pueden multiplicar filas en modo single-query. Hay que revisar el *shape* y valorar `AsSplitQuery()` o una proyección, no aplicar una regla automática.

Composición con filtros dinámicos

Los filtros dinámicos se construyen a partir de expresiones lambda generadas en tiempo de ejecución. Se usan cuando los filtros no se conocen de antemano.

```

var consulta = context.OrdenesFabricacion.AsQueryable();

foreach (var filtro in filtros)
{
    consulta = consulta.Where(filtro);
}

var resultados = consulta.ToList();

```

CSHARP

La primera línea inicia la consulta. La segunda línea itera sobre los filtros. La tercera línea aplica cada filtro. La cuarta línea materializa la consulta. El SQL generado incluye todos los filtros aplicados.

Error común. si se construyen expresiones lambda complejas, el código se vuelve difícil de leer y mantener. Se debe usar esta técnica con moderación.

El problema de la materialización prematura

La materialización prematura ocurre cuando se llama a `ToList`, `FirstOrDefault` o cualquier método que materializa la consulta antes de haber terminado de componerla. Esto provoca que la consulta se ejecute con los filtros aplicados hasta ese momento y que los filtros posteriores se apliquen en memoria.

CSHARP

```
// Anti-patrón: materialización prematura
var ordenes = context.OrdenesFabricacion.ToList();
var filtradas = ordenes.Where(o => o.Estado == "Pendiente").ToList();

// Buen patrón: composición antes de materializar
var filtradas2 = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .ToList();
```

La primera consulta carga todas las órdenes en memoria y después filtra. La segunda consulta filtra en SQL y solo carga las órdenes que cumplen la condición. La segunda es mucho más eficiente.

Error común. si se materializa antes de aplicar los filtros, se transfieren todos los datos de la tabla. Se debe materializar solo al final, después de todos los filtros y ordenaciones.

El problema N+1 en la composición

El problema N+1 puede aparecer si una navegación no se incluyó y Lazy Loading está habilitado: cada primer acceso a una navegación no cargada dentro del bucle puede ejecutar una consulta adicional. Sin Lazy Loading, omitir Include no provoca por sí mismo consultas automáticas.

CSHARP

```
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var planchas = orden.Planchas; // Carga Lazy si está habilitada
}
```

La primera línea carga todas las órdenes. El bucle accede a las planchas de cada orden. Si Lazy Loading está habilitado y la navegación no estaba cargada, cada primer acceso puede ejecutar una consulta adicional. Si Lazy Loading no está habilitado, ese acceso no dispara SQL automáticamente; en AceriaData la colección puede seguir vacía si no se cargó por otra estrategia.

Error común. acceder a navegaciones sin haber definido una estrategia de carga. Con Lazy Loading puede producir consultas adicionales; sin Lazy Loading no se cargan automáticamente. Si la relación se necesita, debe elegirse Eager Loading, Explicit Loading o una proyección adecuada.

El proyecto AceriaData

En el proyecto AceriaData, se añaden métodos de composición de consultas al repositorio de órdenes. Los métodos permiten construir consultas con filtros, ordenaciones y paginación de forma incremental. La capa de aplicación usa estos métodos a través de la unidad de trabajo. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Validación técnica en AceriaData

En el checkpoint M03/PROYECTO0/3.11, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar Database.Migrate(). La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre composición incremental dentro de Infrastructure, filtros opcionales, ordenación dinámica tipada, Skip / Take, ToQueryString() y materialización final. El punto termina con el marcador E2E 3.11 OK.

Resumen de la teoría

- La composición de consultas construye una consulta de forma incremental.
- Cada operador LINQ devuelve un nuevo IQueryable.
- La consulta no se ejecuta hasta que se materializa.
- Los filtros y ordenaciones se aplican de forma condicional.
- Where, OrderBy, ThenBy, Skip, Take y Select se pueden componer.
- La consulta base se puede reutilizar con diferentes filtros.
- Include se puede aplicar de forma condicional.
- La materialización prematura provoca que los filtros se apliquen en memoria.
- El problema N+1 aparece cuando se accede a propiedades de navegación sin Include.

- En el proyecto AceriaData se añaden métodos de composición al repositorio.

Punto 3.12 — Buenas prácticas en el acceso a datos y composición de consultas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se consolidan las buenas prácticas de acceso a datos en el proyecto AceriaData, incluyendo la elección entre carga Eager, Lazy y Explicit, el uso de proyecciones, la composición de consultas y la encapsulación en el repositorio.

Objetivos de aprendizaje

- Comprender las buenas prácticas de acceso a datos con EF Core.
- Elegir entre carga Eager, Lazy y Explicit según el escenario.
- Usar proyecciones para reducir el volumen de datos transferidos.
- Encapsular las consultas en el repositorio para evitar exponer IQueryable.
- Evitar la materialización prematura y el problema N+1.
- Aplicar AsNoTracking en consultas de solo lectura.
- Documentar las decisiones de acceso a datos.
- Aplicar estas buenas prácticas al proyecto AceriaData.

Teoría

Buenas prácticas generales

El acceso a datos con EF Core tiene un conjunto de buenas prácticas que se han ido mencionando a lo largo del curso. Estas prácticas permiten escribir código más eficiente, más mantenible y más predecible. Las más importantes son: usar proyecciones, encapsular las consultas en el repositorio, elegir la técnica de carga adecuada, evitar la materialización prematura, usar AsNoTracking en consultas de solo lectura y documentar las decisiones.

```
// Buen patrón: proyección, filtro y materialización al final
var resúmenes = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea filtra por estado. La tercera línea ordena por fecha. La cuarta línea proyecta al DTO. La quinta línea materializa la consulta. La consulta se ejecuta una sola vez con todos los filtros aplicados.

Elegir la técnica de carga adecuada

La elección entre carga Eager, Lazy y Explicit depende del escenario. Eager Loading resulta apropiado cuando se conocen de antemano las relaciones necesarias y se quiere que los accesos a datos sean explícitos. Lazy Loading puede ser cómodo, pero oculta *roundtrips* y facilita el problema N+1, por lo que debe usarse con especial cuidado. Explicit Loading permite decidir después qué navegación cargar y es útil en flujos condicionales. La elección no depende solo del tipo de aplicación, sino también del número de consultas, el volumen transferido y la previsibilidad del acceso.

```
// Carga Eager: cuando se sabe que se van a necesitar las planchas
var ordenesConPlanchas = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();

// Carga Explicit: cuando se decide bajo demanda
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
if (necesitaPlanchas)
```

CSHARP

```
{
    context.Entry(ordenes).Collection(o => o.Planchas).Load();
}
```

La primera consulta usa carga Eager. La segunda consulta usa carga Explicit condicional. La elección depende del escenario.

Error común. si se usa carga Lazy sin control, el problema N+1 puede degradar el rendimiento. Se debe preferir la carga Eager cuando se sabe que se van a necesitar las entidades relacionadas.

Usar proyecciones

Las proyecciones suelen reducir el volumen de datos transferidos cuando solo se necesita una parte de la entidad. Son una opción preferente para muchos escenarios de lectura, pero el *shape* y el SQL generado deben revisarse en lugar de tratarlas como una regla absoluta.

```
// Sin proyección: carga todas las columnas
var ordenes = context.OrdenesFabricacion.ToList();

// Con proyección: carga solo las columnas necesarias
var resúmenes = context.OrdenesFabricacion
    .Select(o => new { o.NumeroOrden, o.Cliente })
    .ToList();
```

CSHARP

La primera consulta carga todas las columnas. La segunda solicita únicamente las columnas proyectadas. Cuando esas son realmente las únicas necesarias, la proyección reduce el volumen transferido; el beneficio concreto depende del tamaño de la entidad, del número de filas y del *shape* de la consulta.

Error común. si se proyecta a un tipo anónimo y después se necesita la entidad completa, se debe ejecutar una consulta adicional. Se debe proyectar solo cuando se sabe que no se va a necesitar la entidad completa.

Encapsular las consultas en el repositorio

Las consultas se deben encapsular en métodos del repositorio para evitar exponer IQueryable fuera de la infraestructura. Esto permite controlar qué consultas se ejecutan y cómo se componen.

```
public interface IOrdenRepositorio
{
    List<OrdenResumenDto> ObtenerResúmenesPendientes();
    List<OrdenResumenDto> ObtenerResúmenesPorCliente(string cliente);
    int ContarOrdenesPendientes();
}
```

CSHARP

La primera línea declara la interfaz. La segunda línea declara el método que obtiene los resúmenes pendientes. La tercera línea declara el método que obtiene los resúmenes por cliente. La cuarta línea declara el método que cuenta las órdenes pendientes.

Error común. si el repositorio devuelve IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Esto rompe la abstracción. Se deben devolver listas o valores concretos.

Evitar la materialización prematura

La materialización prematura ocurre cuando se llama a ToList antes de haber terminado de componer la consulta. Esto provoca que los filtros posteriores se apliquen en memoria.

```
// Anti-patrón: materialización prematura
var ordenes = context.OrdenesFabricacion.ToList();
var filtradas = ordenes.Where(o => o.Estado == "Pendiente").ToList();

// Buen patrón: composición antes de materializar
var filtradas2 = context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .ToList();
```

CSHARP

La primera consulta carga todas las órdenes y después filtra. La segunda consulta filtra en SQL y solo carga las órdenes que cumplen la condición.

Error común. si se materializa antes de aplicar los filtros, se transfieren todos los datos de la tabla. Se debe materializar solo al final.

Evitar el problema N+1

El problema N+1 aparece cuando una consulta obtiene las entidades principales y después se ejecuta una consulta adicional por cada una para obtener datos relacionados. Con EF Core suele aparecer, por ejemplo, al acceder en un bucle a navegaciones Lazy todavía no cargadas.

```
// Anti-patrón: N+1
var ordenes = context.OrdenesFabricacion.ToList();
foreach (var orden in ordenes)
{
    var planchas = orden.Planchas;
}

// Buen patrón: una sola consulta
var ordenesConPlanchas = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .ToList();
```

CSHARP

Con Lazy Loading habilitado y las navegaciones inicialmente sin cargar, el primer patrón puede ejecutar N+1 consultas. El segundo expresa Eager Loading; con una sola colección y el modo single-query usado en este ejemplo se resuelve con una consulta.

Error común. con Lazy Loading habilitado, si la navegación aún no está cargada, acceder a ella dentro de un bucle puede disparar una consulta adicional por entidad y producir N+1. Si se sabe que la relación será necesaria, conviene planificar la carga Eager o una proyección adecuada.

Usar AsNoTracking en consultas de solo lectura

El método AsNoTracking carga las entidades sin registrarlas en el Change Tracker. Es útil para consultas de solo lectura donde no se van a modificar los datos.

```
// Con Tracking: las entidades se registran en el Change Tracker
var ordenesConTracking = context.OrdenesFabricacion.ToList();

// Sin Tracking: las entidades no se registran en el Change Tracker
var ordenesSinTracking = context.OrdenesFabricacion
    .AsNoTracking()
    .ToList();
```

CSHARP

La primera consulta usa tracking y la segunda no. En lecturas donde no se van a modificar entidades, `AsNoTracking()` evita el coste del Change Tracker; el beneficio concreto depende del volumen, la duplicación de entidades y la necesidad de resolución de identidad.

Error común. si se usa AsNoTracking y después se modifica una entidad, los cambios no se guardan porque la entidad no está registrada en el Change Tracker. Se debe usar AsNoTracking solo en consultas de solo lectura.

Usar AsNoTracking con proyecciones

`AsNoTracking()` es relevante cuando el resultado contiene entidades que, de otro modo, EF Core rastrearía. Una proyección formada únicamente por valores escalares o por un DTO sin instancias de entidad no añade entidades al Change Tracker; si una proyección incluye una instancia de entidad, esa entidad sí puede rastrearse salvo que la consulta sea no-tracking.

```
var resmenes = context.OrdenesFabricacion
    .AsNoTracking()
    .Select(o => new OrdenResumenDto
    {
        NumeroOrden = o.NumeroOrden,
        Cliente = o.Cliente,
        Estado = o.Estado
    })
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea aplica `AsNoTracking`. La tercera línea proyecta al DTO. La cuarta línea materializa la consulta.

Error común. si se aplica `AsNoTracking` a una consulta que devuelve entidades completas y después se modifican, los cambios no se guardan. Se debe usar `AsNoTracking` solo cuando no se van a modificar las entidades.

Evaluar `AsSplitQuery` para cargas relacionadas

`AsSplitQuery()` divide la carga de navegaciones de colección incluidas en varias consultas SQL coordinadas. Es especialmente útil para evitar la explosión cartesiana causada por colecciones hermanas cargadas al mismo nivel, aunque introduce *roundtrips* adicionales y no es una mejora automática para cualquier consulta.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

CSHARP

La primera línea inicia la consulta. La segunda línea incluye la colección de planchas. La tercera línea incluye la colección de entidades intermedias. La cuarta línea divide la consulta en varias. La quinta línea materializa la consulta.

Error común. convertir `AsSplitQuery()` en una regla automática. Puede ser útil incluso con una colección grande, y también puede ser peor por los *roundtrips* adicionales. Se debe decidir según la forma y el volumen de la consulta.

Usar `FirstOrDefault` en lugar de `First` cuando puede no haber resultados

El método `FirstOrDefault` devuelve null si no hay resultados. El método `First` lanza una excepción. Se debe usar `FirstOrDefault` cuando no se sabe si hay resultados.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
if (orden is not null)
{
    // Procesar la orden
}
```

CSHARP

La primera línea busca la orden por Id y devuelve null si no existe. La segunda línea comprueba si la orden existe antes de procesarla.

Error común. si se usa `First` y no hay resultados, se lanza una excepción. Se debe usar `FirstOrDefault` cuando puede no haber resultados.

Usar `Any` en lugar de `Count` cuando solo se quiere saber si hay elementos

`Any()` expresa una consulta de existencia y EF Core normalmente la traduce a `EXISTS`; `Count()` solicita un recuento. Si solo interesa saber si existe alguna fila, `Any()` evita pedir un total que no se necesita.

```
// Buen patrón: Any se detiene en el primer elemento
var existe = context.OrdenesFabricacion.Any(o => o.Estado == "Pendiente");

// Alternativa menos directa para una comprobación de existencia
var existe2 = context.OrdenesFabricacion.Count(o => o.Estado == "Pendiente") > 0;
```

CSHARP

La primera consulta expresa directamente existencia; la segunda solicita un recuento completo. Para este objetivo, `Any()` comunica mejor la intención y evita calcular un total innecesario.

Error común. usar `Count() > 0` cuando solo se necesita una comprobación de existencia. EF Core puede traducir `Any()` a `EXISTS`, que expresa directamente la intención sin solicitar un recuento completo.

Documentar las decisiones de acceso a datos

Las decisiones de acceso a datos deben documentarse en el código. Esto permite que otros desarrolladores entiendan por qué se eligió una técnica concreta.

```

/// <summary>
/// Obtiene los resúmenes de las órdenes pendientes.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado
        })
        .ToList();
}

```

CSHARP

La primera línea inicia el comentario XML. La segunda línea describe el método. La tercera línea explica el uso de la proyección. La cuarta línea explica el uso de AsNoTracking. La quinta línea cierra el comentario. La sexta línea declara el método. La séptima línea inicia la consulta. La octava línea aplica AsNoTracking. La novena línea filtra por estado. La décima línea proyecta al DTO. La undécima línea materializa la consulta.

Error común. si no se documentan las decisiones, otros desarrolladores pueden modificarlas sin entender las consecuencias. Se deben documentar las decisiones de acceso a datos.

El proyecto AceriaData

En el proyecto AceriaData, se aplican las buenas prácticas de acceso a datos a todos los métodos del repositorio. Se usan proyecciones, AsNoTracking, AsSplitQuery, Any y FirstOrDefault. Se encapsulan las consultas en métodos del repositorio. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Validación técnica en AceriaData

El cierre 3.12 retira del modelo el `AutoInclude()` introducido con fines docentes en 3.8. Igual que ocurre con el seam `IQueryable`, la capacidad se ha aprendido y probado, pero no se conserva como comportamiento implícito del contrato final. La demostración de `AsSplitQuery()` sí se conserva y carga dos colecciones hermanas (`Planchas` y `OrdenesAleaciones`) más `Detalle`, de modo que la razón para dividir la consulta es observable.

En el checkpoint M03/PROYECTO/3.12, esta materia se ejecuta sobre SQL Server LocalDB después de aplicar `Database.Migrate()`. La demostración usa datos deterministas, limpia el Change Tracker antes de consultar y falla si el resultado real no coincide con lo esperado.

La implementación validada cubre `AsNoTracking()`, `Any()` para existencia, proyecciones de lectura, `AsSplitQuery()` cuando la forma lo justifica, `FirstOrDefault()`, el contrato final deja de exponer `IQueryable`. El punto termina con el marcador E2E 3.12 OK.

Resumen de la teoría

- Las buenas prácticas de acceso a datos mejoran el rendimiento y la mantenibilidad.
- La elección entre carga Eager, Lazy y Explicit depende del escenario.
- Las proyecciones reducen el volumen de datos transferidos.
- Las consultas se encapsulan en el repositorio para evitar exponer IQueryable.
- La materialización prematura provoca que los filtros se apliquen en memoria.
- El problema N+1 se evita diseñando explícitamente la estrategia de carga; `Include` es una opción cuando se necesita Eager Loading.
- `AsNoTracking` se usa en consultas de solo lectura.
- `AsSplitQuery()` se evalúa cuando la forma de carga relacionada puede beneficiarse de dividir la consulta; en `AceriaData` se usa con dos colecciones hermanas.
- `FirstOrDefault` se usa cuando puede no haber resultados.
- `Any` se usa cuando solo se quiere saber si hay elementos.
- Las decisiones de acceso a datos se documentan.
- En el proyecto `AceriaData` se aplican estas buenas prácticas a todos los métodos.